

COMSATS UNIVERSITY ISLAMABAD

WAH CAMPUS

Department of Computer Science

DevOps Assignment

Course Title: DevOps

Course Code: CSC323

Instructor Name: Sir Ibrahim

Class: BSE-8A

Name	Registration No.
Muhammad Sami	SP23-BSE-039

1. Objective

The objective of this lab task is to build a Kubernetes cluster with Minikube on an Ubuntu virtual machine running on AWS EC2, to deploy and manage several applications on that cluster, and then to extend the original single-node cluster to a two-node logical cluster. The practical covers launching and preparing the EC2 instance, installing Docker, kubectl and Minikube, deploying Nginx first with kubectl commands and then with YAML manifests, deploying a two-tier WordPress and MySQL application, and deploying a Django to-do list application, while working with Kubernetes Deployments, Pods, Services, replicas and self-healing. In Part 2, a worker node, `minikube-m02`, is added to the original Minikube node `minikube`. The three replicas of the to-do deployment are scheduled on the worker node, while the existing Nginx and WordPress workloads stay on the original node. The lab ends by verifying node readiness, pod placement, application availability and the recovery of a deleted pod by Kubernetes. Both Kubernetes nodes are Minikube nodes that run as Docker containers on the same EC2 instance; they are not separate EC2 instances.

2. Lab Environment

The table below summarises the environment used in this lab. All values are taken from the screenshots in this report. Every command in the report is run over SSH on the EC2 instance as the `ubuntu` user (prompt `ubuntu@ip-172-31-38-26`).

Item	Details
Cloud platform	Amazon Web Services (AWS) EC2, region eu-north-1 (Availability Zone eu-north-1b)
Instance name / ID	sami_kubernetes_lab / i-0e2030e9ad8ded0d6
Instance type	m7i-flex.large (2 vCPU, 8 GiB memory)
Operating system	Ubuntu (selected from the AMI Quick Start list; Minikube reports Ubuntu 26.04)
Storage	30 GiB gp3 root volume
IP addresses	Public IPv4: 16.171.154.62; Private IPv4: 172.31.38.26
Docker	Version 29.1.3 (installed from the docker.io package)
Minikube	v1.39.0 (Docker driver)
kubectl	Client v1.35.9 (Kustomize v5.7.1)
Kubernetes cluster	v1.37.0, single node “minikube” (containerd 2.3.4)

Note: this report has two parts. Part 1 (Sections 1 to 12) was done on a single-node Minikube cluster, and the table above describes that environment. Part 2 (Sections 13 to 19) is the work I did afterwards: I extended the same cluster to two logical nodes by adding the worker node minikube-m02 on the same EC2 instance.

3. Launching the AWS EC2 Instance

The first activity is creating the virtual server that hosts the Kubernetes cluster. The instance is configured in the AWS EC2 "Launch an instance" wizard. In real-world DevOps work, a cloud virtual machine like this is a quick way to obtain a clean, disposable Linux server for building and testing infrastructure.

3.1 Instance Name and Operating System

In this step I start the EC2 launch wizard, give the instance the name `sami_kubernetes_lab` and choose the operating system image (AMI) for the server. Ubuntu is selected from the Quick Start tab, so the server will run Ubuntu Linux.

Settings shown

- **Name** – `sami_kubernetes_lab`, the label used to identify the instance in the AWS console.
- **Application and OS Images (Amazon Machine Image)** – an AMI contains the operating system and software the instance starts with. The **Ubuntu** tile is highlighted under Quick Start.

Observation

The name field contains `sami_kubernetes_lab` and the Ubuntu option is highlighted, confirming the operating system choice.

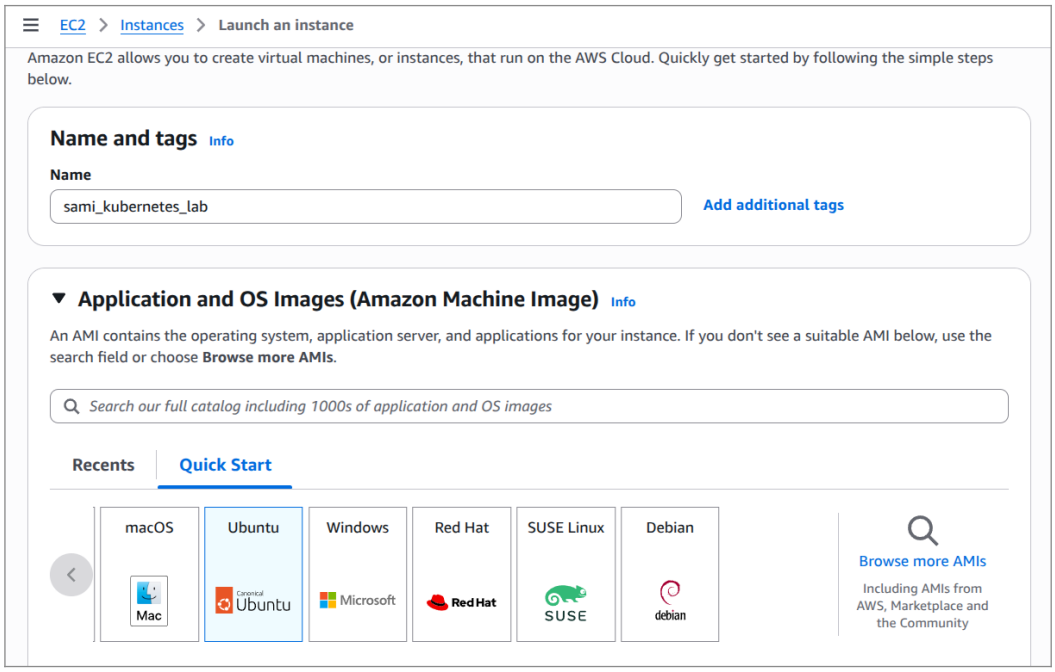


Figure 1: *Launch wizard showing the instance name and the Ubuntu AMI selection*

3.2 Storage Configuration

I next set the size and type of the disk attached to the server. Minikube downloads container images and Kubernetes components, so the root disk needs enough free space.

Settings shown

- **1x 30 GiB** – one root volume with a size of 30 GiB.
- **gp3** – the general-purpose SSD volume type.

- **Root volume, 3000 IOPS, Not encrypted** — the properties AWS displays for the volume.

Observation

The Configure storage panel shows a 30 GiB gp3 root volume.

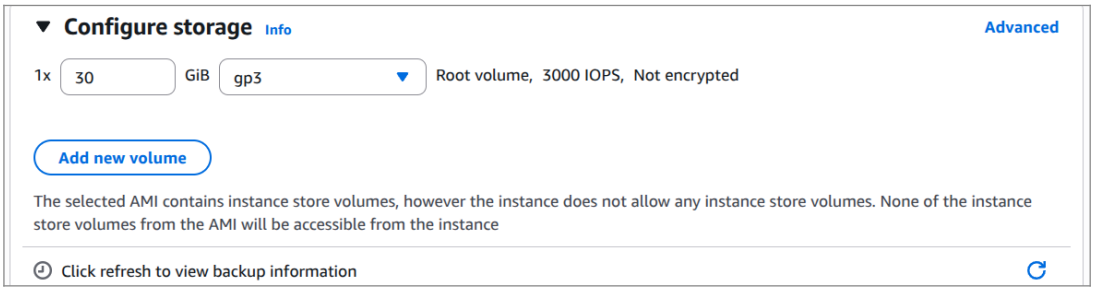


Figure 2: *Configure storage: 30 GiB gp3 root volume*

3.3 Instance Type

I then choose the hardware size of the virtual machine. The Kubernetes cluster runs inside this single server, so it needs a reasonable amount of CPU and memory.

Settings shown

- **m7i-flex.large** — the selected instance type, labelled "Free tier eligible".
- **2 vCPU, 8 GiB Memory** — the compute and memory provided by this instance type.

Observation

The Instance type panel shows m7i-flex.large with 2 vCPU and 8 GiB of memory, together with the on-demand pricing information for each operating system.

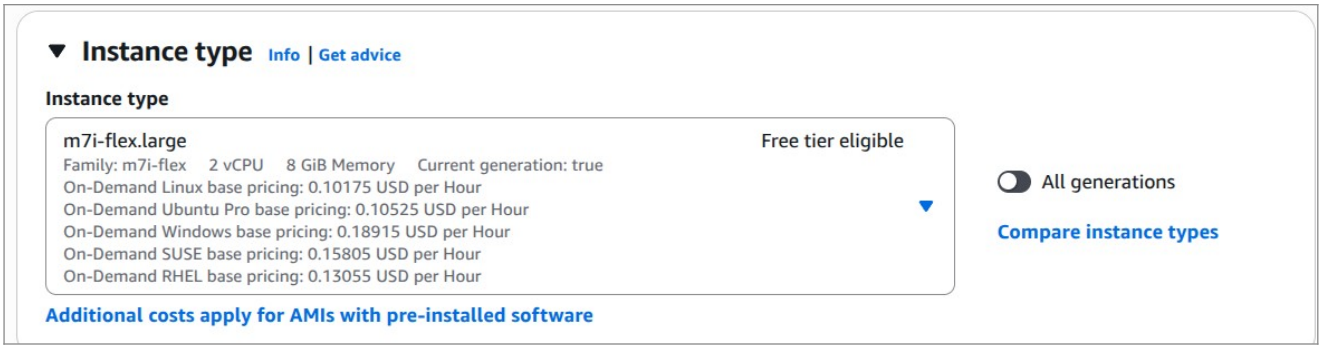


Figure 3: *Instance type selection: m7i-flex.large (2 vCPU, 8 GiB memory)*

3.4 Instance Summary After Launch

After the wizard is completed, the instance appears in the EC2 console. I open its summary page to note the details needed in later steps. In particular, the public IPv4 address is used for SSH access and for opening the deployed applications in a web browser.

Details shown

- **Instance ID** — `i-0e2030e9ad8ded0d6` (name `sami_kubernetes_lab`).
- **Instance state** — Running.

- **Public IPv4 address** – `16.171.154.62`, with the public DNS name `ec2-16-171-154-62.eu-north-1.compute.amazonaws.com`.
- **Private IPv4 address** – `172.31.38.26`, which matches the hostname `ip-172-31-38-26` seen in the terminal prompt later in the report.
- **Instance type** – `m7i-flex.large`.

Observation

The instance is in the Running state and has both a public and a private IP address.

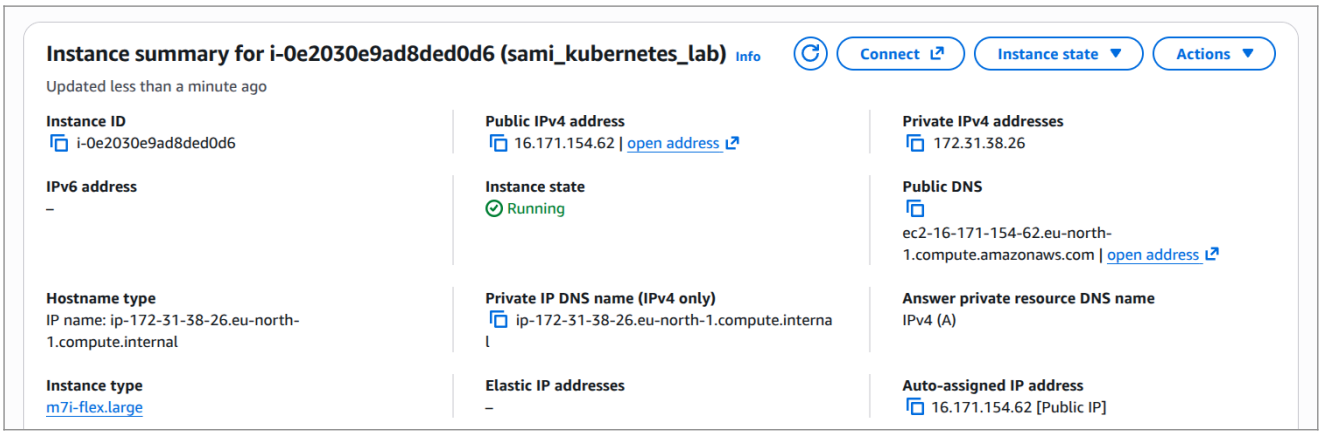


Figure 4: *EC2 instance summary showing the running instance and its IP addresses*

3.5 Security Group Inbound Rules

For the server to be reachable, its security group (the AWS virtual firewall) must allow the required ports. I review the inbound rules attached to the instance. SSH is needed to manage the server, and the extra TCP ports are used later to open the Nginx, WordPress and to-do applications from a browser.

Rules shown

- **SSH, TCP, port 22** – remote terminal access to the server.
- **Custom TCP, ports 8082, 8083, 8081 and 8080** – ports later used for the applications (see the port-forward commands in Sections 7 to 10).

Observation

The Inbound rules list contains five IPv4 rules: port 22 (SSH) and the custom TCP ports 8082, 8083, 8081 and 8080.

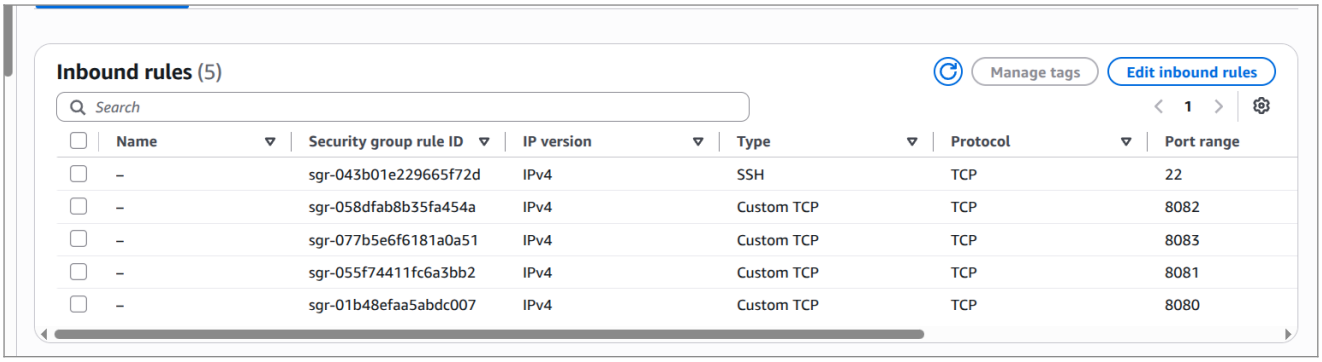


Figure 5: *Security group inbound rules for SSH and application ports 8080–8083*

4. Preparing the Server and Installing Docker

After connecting to the instance over SSH, I prepare the Ubuntu server. Minikube is started with the Docker driver in this lab, so Docker must be installed first.

4.1 Updating the Package Index

Before installing software, I refresh the list of packages available from the Ubuntu repositories. This makes sure the latest package versions and dependency information are used.

Command used

```
sudo apt update
```

Command explanation

- `sudo` — runs the command with administrator (root) privileges.
- `apt update` — downloads the latest package lists from the configured repositories. It does not install or upgrade any package.

Result

The terminal shows `Hit` and `Get` lines for the Ubuntu repositories on `eu-north-1.ec2.archive.ubuntu.com` and `security.ubuntu.com`, which means the package lists are being fetched. The captured part of the output shows the repository downloads; the next screenshot shows that 193 packages can be upgraded.



Figure 6: *Updating the package index with sudo apt update*

4.2 Installing Docker

I install Docker from the Ubuntu repository. Docker provides the container runtime that Minikube uses to create the Kubernetes node.

Command used

```
sudo apt install docker.io -y
```

Command explanation

- `apt install docker.io` — installs the `docker.io` package, which is Ubuntu's Docker engine.
- `-y` — automatically answers "yes" to the confirmation prompt so the installation continues without waiting for input.

Result

The output starts with the line "193 packages can be upgraded". It then lists `docker.io` as the package to install together with its dependencies (`bridge-utils`, `containerd`, `dns-root-data`, `dnsmasq-base`, `pigz`, `runc` and `ubuntu-fan`). The summary reports 8 packages to install and a download size of 74.0 MB.

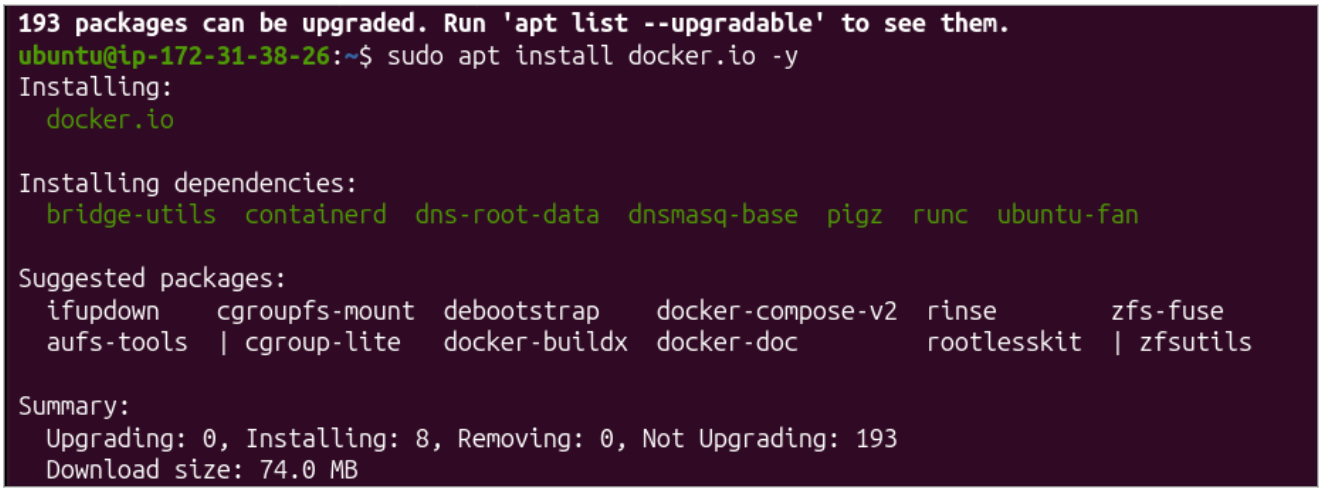


Figure 7: *Installing Docker with sudo apt install docker.io -y*

4.3 Enabling and Verifying Docker

I make sure the Docker service is running and check that the installation works. This step also shows a permission error that is fixed later in the lab.

Commands used

```
sudo systemctl enable --now docker
docker --version
docker ps
sudo docker ps
```

Command explanation

- `sudo systemctl enable --now docker` — `enable` makes the Docker service start automatically at boot and `--now` starts it immediately.
- `docker --version` — prints the installed Docker version.
- `docker ps` — lists the running containers.
- `sudo docker ps` — the same listing, run with administrator privileges.

Result

Docker reports version 29.1.3 (build 29.1.3-0ubuntu4.1), so it is installed. Running `docker ps` as the normal user fails with "permission denied while trying to connect to the docker API at unix:///var/run/docker.sock",

meaning the `ubuntu` user is not yet allowed to talk to the Docker daemon. Running the same command with `sudo` works and shows only the column headings, so Docker is running and no containers exist yet. The permission problem is solved in Section 6.1 by adding the user to the `docker` group.

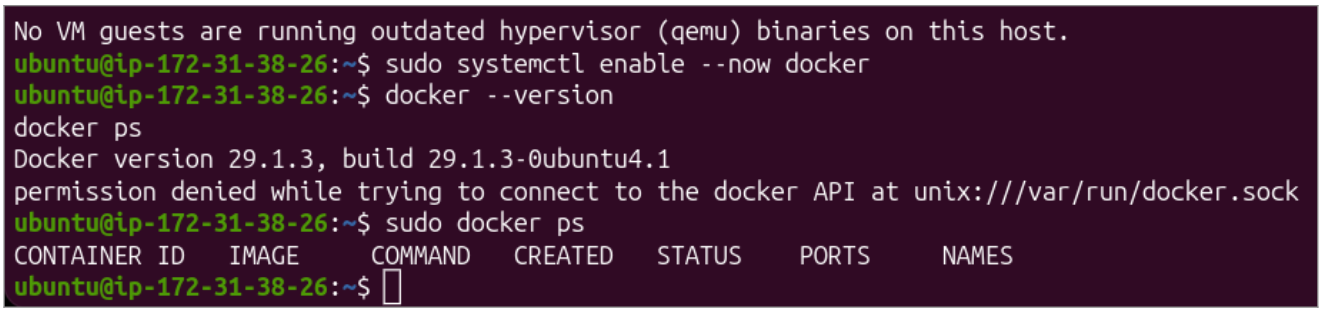


Figure 8: *Docker enabled and verified; docker ps fails without sudo and works with sudo*

5. Installing Minikube and kubectl

Minikube creates a local single-node Kubernetes cluster, and kubectl is the command-line tool used to control it. Both are installed on the EC2 server in this section.

5.1 Downloading Minikube

I download the Minikube program for Linux from the official Minikube release storage.

Command used

```
curl -LO https://storage.googleapis.com/minikube/releases/latest/minikube-linux-amd64
```

Command explanation

- `curl` – a tool for transferring data from a URL.
- `-L` – follows redirects if the server moves the file.
- `-O` – saves the file in the current directory using its original name, `minikube-linux-amd64`.

Result

The curl progress table reaches 100% with 136.0 MB received. The transfer took about 4 seconds at an average speed of 29.58 MB/s.

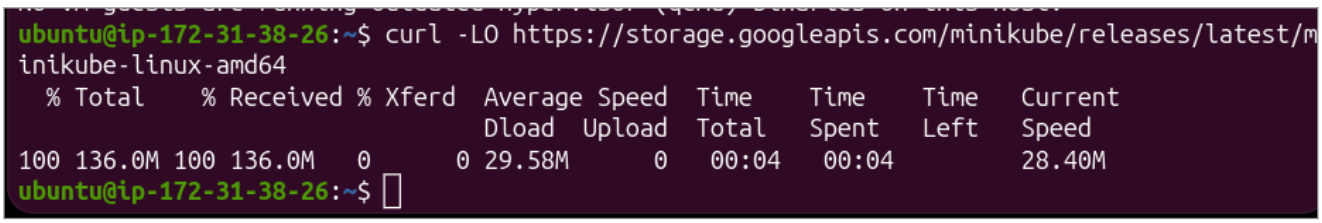


Figure 9: *Downloading the Minikube binary with curl*

5.2 Installing Minikube and Checking Its Version

The downloaded file is installed into a system folder so that the `minikube` command can be run from any location. I then confirm the installation by printing the version.

Commands used

```
sudo install minikube-linux-amd64 /usr/local/bin/minikube
minikube version
```

Command explanation

- `sudo install minikube-linux-amd64 /usr/local/bin/minikube` → copies the downloaded file to `/usr/local/bin` under the name `minikube` and makes it executable.
- `minikube version` → prints the installed Minikube version.

Result

The output reports `minikube version: v1.39.0` together with its commit ID, which shows Minikube is installed and available as a command.



Figure 10: *Installing Minikube to /usr/local/bin and checking the version*

5.3 Checking the Instance Status in the AWS Console

While the tools are being installed, I return to the EC2 instance list in the AWS console to confirm that the server is healthy.

Details shown

- **Instance state** → Running.
- **Status check** → 3/3 checks passed.
- **Availability Zone** → `eu-north-1b`.

Observation

The instance `sami_kubernetes_lab` (`m7i-flex.large`) is running and all three status checks have passed.



Figure 11: *EC2 instance list showing the running instance with 3/3 status checks passed*

5.4 Installing kubectl

I install kubectl, the tool used to send commands to the Kubernetes cluster, and verify the client version.

Commands used

```
sudo snap install kubectl --classic
kubectl version --client
```

Command explanation

- `sudo snap install kubectl --classic` → installs kubectl from the Snap store. `--classic` installs it with classic confinement, giving it normal access to the system and to the user's files.

- `kubectl version --client` — prints only the client version and does not contact the cluster.

Result

Snap reports that `kubectl 1.35.9 from Canonical` is installed. The version command shows Client Version v1.35.9 and Kustomize Version v5.7.1.

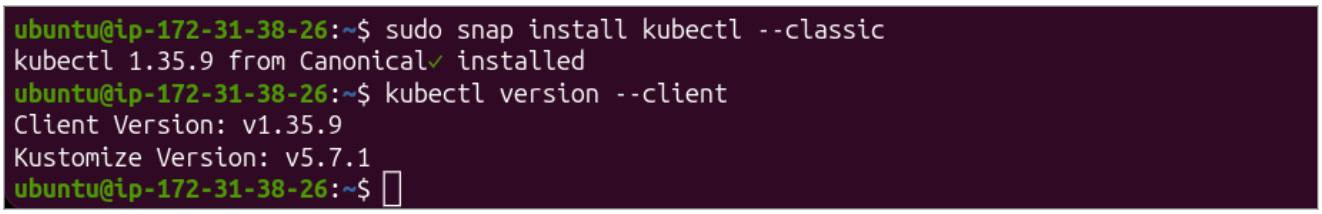


Figure 12: *Installing kubectl with Snap and checking the client version*

5.5 Installing curl

I install the `curl` package using apt, so that the transfer tool used earlier is installed and up to date on the server.

Command used

```
sudo apt install curl -y
```

Command explanation

- `apt install curl` — installs the `curl` package (or upgrades it if it is already present).
- `-y` — answers "yes" automatically.

Result

The output lists `curl`, `libcurl3t64-gnutls` and `libcurl4t64` under "Upgrading". The summary shows 3 packages upgraded and 0 newly installed, with a download size of 1120 kB, which means curl was already present and has now been updated.

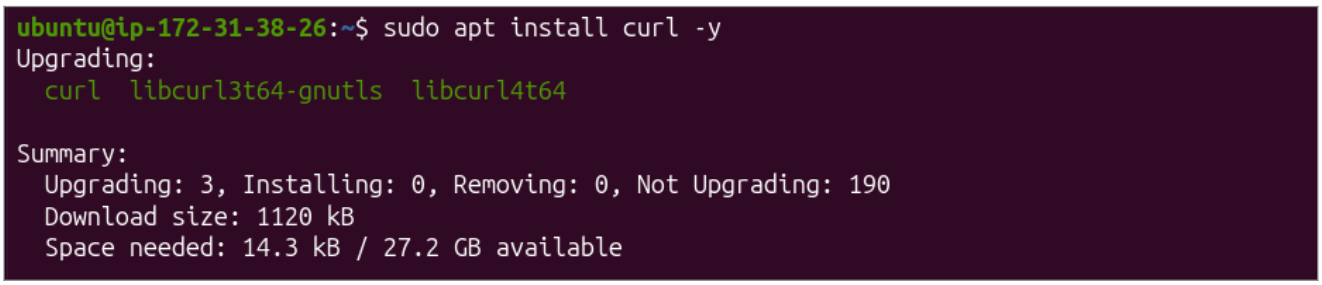


Figure 13: *Installing and updating curl with apt*

6. Starting the Kubernetes Cluster

6.1 Starting Minikube with the Docker Driver

Before starting Minikube, I fix the Docker permission error seen in Section 4.3 by adding the `ubuntu` user to the `docker` group. Minikube is then started with Docker as its driver. In real-world environments, tools such as Minikube are used to run a small Kubernetes cluster for development and testing before using a production cluster.

Commands used

```
sudo usermod -aG docker ubuntu
newgrp docker
docker ps
minikube start --driver=docker
```

Command explanation

- `sudo usermod -aG docker ubuntu` → `-G docker` sets the `docker` group and `-a` appends it, so the `ubuntu` user keeps its existing groups.
- `newgrp docker` → starts a shell in which the new group membership is active immediately, without logging out.
- `docker ps` → checks that Docker can now be used without `sudo`.
- `minikube start --driver=docker` → creates the Kubernetes cluster; `--driver=docker` makes Minikube run the Kubernetes node as a Docker container.

Result

`docker ps` now runs without `sudo` and shows only the column headings, so the earlier permission error is resolved. Minikube v1.39.0 starts on Ubuntu 26.04 using the Docker driver, begins the "minikube" control-plane node, pulls the base image and downloads the Kubernetes v1.37.0 preload. The screenshot was captured while the base-image download was at 74.27%; the completed output is shown next.

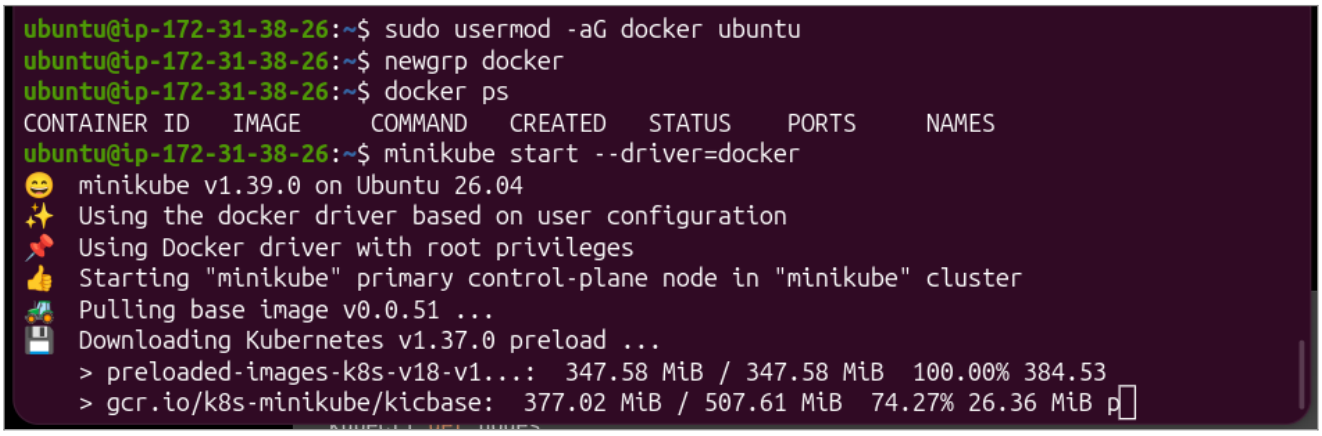


Figure 14: *Adding the user to the docker group and starting Minikube (download in progress)*

6.2 Minikube Start Completed

The same `minikube start --driver=docker` command is shown after it has finished. This output lists each stage of the cluster creation.

Command used

```
minikube start --driver=docker
```

Command explanation

- `minikube start` → creates and starts the local Kubernetes cluster.
- `--driver=docker` → uses Docker to host the cluster node.

Result

Both downloads reach 100%. Minikube creates a Docker container (CPUs=2, Memory=3072MB), prepares Kubernetes v1.37.0 on containerd 2.3.4, configures the CNI (Container Networking Interface) and verifies the

Kubernetes components. The addons `default-storageclass` and `storage-provisioner` are enabled. A notice says that kubectl 1.35.9 may have incompatibilities with Kubernetes 1.37.0; it is a version-difference warning, and the output ends with “Done! kubectl is now configured to use ‘minikube’ cluster and ‘default’ namespace by default”.



Figure 15: *Minikube cluster creation completed successfully*

6.3 Verifying the Cluster

To confirm that the cluster is healthy, I check the Minikube status, the Kubernetes node and the cluster information.

Commands used

```
minikube status
kubectl get nodes
kubectl cluster-info
```

Command explanation

- `minikube status` – shows whether the Minikube components are running.
- `kubectl get nodes` – lists the nodes in the Kubernetes cluster.
- `kubectl cluster-info` – shows the addresses of the control plane and core services.

Result

Minikube reports `type: Control Plane` with host, kubelet and apiserver all `Running` and kubeconfig `Configured`. The node `minikube` has status `Ready`, role `control-plane` and version `v1.37.0`. The cluster information shows the control plane at `https://192.168.49.2:8443` and CoreDNS running. The Kubernetes cluster is therefore ready for deployments.

```
ubuntu@ip-172-31-38-26:~$ minikube status
minikube
type: Control Plane
host: Running
kubelet: Running
apiserver: Running
kubeconfig: Configured

ubuntu@ip-172-31-38-26:~$ kubectl get nodes
NAME       STATUS   ROLES           AGE    VERSION
minikube   Ready    control-plane   2m5s   v1.37.0
ubuntu@ip-172-31-38-26:~$ kubectl cluster-info
Kubernetes control plane is running at https://192.168.49.2:8443
CoreDNS is running at https://192.168.49.2:8443/api/v1/namespaces/kube-system/services/kube-
dns:dns/proxy

To further debug and diagnose cluster problems, use 'kubectl cluster-info dump'.
ubuntu@ip-172-31-38-26:~$
```

Figure 16: *Verifying the cluster with minikube status, kubectl get nodes and kubectl cluster-info*

7. Deploying Nginx Using kubectl Commands

The first application is deployed with direct (imperative) kubectl commands. This is the quickest way to create a Deployment and expose it, and it is useful for learning and testing.

7.1 Creating and Exposing the Nginx Deployment

I create an Nginx deployment and then expose it as a service so that it can be reached from outside its pod.

Commands used

```
kubectl create deployment nginx --image=nginx:latest
kubectl expose deployment nginx --type=NodePort --port=80
```

Command explanation

- `kubectl create deployment nginx` – creates a Deployment named `nginx`, which keeps the required pod running.
- `--image=nginx:latest` – the container image to run, here the latest Nginx image.
- `kubectl expose deployment nginx` – creates a Service for the deployment.
- `--type=NodePort` – opens a port on the node so the service can be reached from outside the cluster.
- `--port=80` – the port on which the service listens.

Result

Kubernetes replies `deployment.apps/nginx created` and `service/nginx exposed`, so both objects were created successfully.

```
ubuntu@ip-172-31-38-26:~$ kubectl create deployment nginx --image=nginx:latest
deployment.apps/nginx created
ubuntu@ip-172-31-38-26:~$ kubectl expose deployment nginx --type=NodePort --port=80
service/nginx exposed
ubuntu@ip-172-31-38-26:~$
```

Figure 17: *Creating the nginx deployment and exposing it as a NodePort service*

7.2 Verifying the Deployment, Service and Pod

I check that Kubernetes has created the objects and that the pod is running.

Commands used

```
kubectl get deployment nginx
kubectl get service nginx
kubectl get pods
```

Command explanation

- `kubectl get deployment nginx` – shows the deployment status. `READY 1/1` means 1 of the 1 desired replicas is ready.
- `kubectl get service nginx` – shows the service type, internal address and ports.
- `kubectl get pods` – lists the pods and their state.

Result

The deployment `nginx` shows `1/1` ready and available. The service `nginx` is of type `NodePort` with cluster IP `10.111.198.135` and ports `80:30131/TCP`, meaning service port 80 is mapped to node port 30131. The pod `nginx-598cc96cd9-jhf89` is `1/1 Running` with 0 restarts.

```
ubuntu@ip-172-31-38-26:~$ kubectl get deployment nginx
NAME    READY   UP-TO-DATE   AVAILABLE   AGE
nginx   1/1     1            1           76s
ubuntu@ip-172-31-38-26:~$ kubectl get service nginx
NAME    TYPE       CLUSTER-IP       EXTERNAL-IP   PORT(S)        AGE
nginx   NodePort   10.111.198.135   <none>        80:30131/TCP   72s
ubuntu@ip-172-31-38-26:~$ kubectl get pods
NAME                     READY   STATUS    RESTARTS   AGE
nginx-598cc96cd9-jhf89   1/1     Running   0          104s
ubuntu@ip-172-31-38-26:~$
```

Figure 18: *Nginx deployment, NodePort service and pod verified with kubectl get*

7.3 Accessing Nginx from the Browser

The Minikube cluster runs on its own internal network address (`192.168.49.2`, shown in Section 6.3), so I use port forwarding to make the service reachable through the EC2 public IP address. The command keeps running in the terminal while the forward is active.

Command used

```
kubectl port-forward service/nginx 8082:80 --address=0.0.0.0
```

Command explanation

- `kubectl port-forward` – forwards a port on the server to a port of a Kubernetes service.
- `service/nginx` – the service to forward to.
- `8082:80` – server port 8082 is forwarded to port 80 of the service.
- `--address=0.0.0.0` – listens on all network interfaces, not only on localhost, so the application can be opened through the public IP.

Result

The terminal prints `Forwarding from 0.0.0.0:8082 -> 80` and stays active without returning to the prompt, which shows that the port forward is running.

```
ubuntu@ip-172-31-38-26:~$ kubectl port-forward service/nginx 8082:80 --address=0.0.0.0
Forwarding from 0.0.0.0:8082 -> 80
```

Figure 19: *Port-forwarding the nginx service to port 8082*

With the port forward running, I open the EC2 public IP address on port 8082 in a web browser to test the application.

Address opened in the browser

```
16.171.154.62:8082
```

Address explanation

- `16.171.154.62` — public IPv4 address of the EC2 instance.
- `8082` — the forwarded port that reaches the Nginx service.

Observation

The browser displays the default "Welcome to nginx!" page, confirming that Nginx is running in Kubernetes and is reachable from the internet through port 8082.

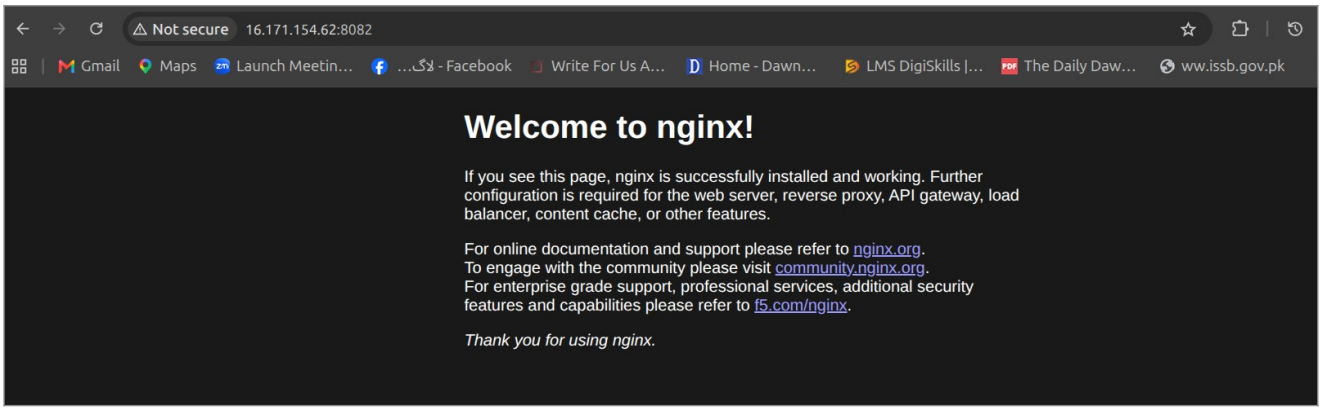


Figure 20: *Nginx welcome page opened at 16.171.154.62:8082*

7.4 Re-checking the Resources After Some Time

Later in the lab, I repeat the same three `kubectl get` commands to check that the Nginx resources are still healthy.

Commands used

```
kubectl get deployment nginx
kubectl get service nginx
kubectl get pods
```

Command explanation

- The commands are the same as in Section 7.2 and show the deployment, service and pods of `nginx`.

Result

The deployment is still `1/1` ready, the service is still `NodePort` (`80:30131/TCP`) and the pod `nginx-598cc96cd9-jhf89` is still `Running` with 0 restarts. The ages (about 9 minutes) show that the application has been running stably.

```
ubuntu@ip-172-31-38-26:~$ kubectl get deployment nginx
NAME    READY   UP-TO-DATE   AVAILABLE   AGE
nginx   1/1     1            1           9m11s
ubuntu@ip-172-31-38-26:~$ kubectl get service nginx
NAME    TYPE       CLUSTER-IP       EXTERNAL-IP   PORT(S)        AGE
nginx   NodePort   10.111.198.135   <none>        80:30131/TCP   9m7s
ubuntu@ip-172-31-38-26:~$ kubectl get pods
NAME                     READY   STATUS    RESTARTS   AGE
nginx-598cc96cd9-jhf89   1/1     Running   0          9m33s
ubuntu@ip-172-31-38-26:~$
```

Figure 21: *Nginx deployment, service and pod still running after several minutes*

8. Deploying Nginx Using YAML Manifests

The next deployment uses YAML manifest files. This declarative approach describes the desired state of the application in files, which can be stored, reviewed and reused. In real-world DevOps workflows, manifests like these are normally kept in version control.

8.1 Creating the Deployment Manifest

I write the Deployment manifest `nginx-dep.yml` in the nano text editor (the editing commands are shown in Section 8.3). The file asks Kubernetes to run four copies of an Nginx container.

Key fields in the file

- `kind: Deployment` and `name: nginx-dep` → the type and name of the resource.
- `replicas: 4` → four identical pods must be kept running.
- `selector.matchLabels` → `app: nginx-kube`; the deployment manages pods that carry this label.
- `template.metadata.labels` → `app: nginx-kube`; the label given to every pod created from the template.
- `containers` → one container named `nginx` using the image `nginx:1.14.2` and `containerPort: 80`.

Observation

The editor shows the complete manifest, ending with `containerPort: 80`. The title bar shows `nginx-dep.yml`, and the asterisk means the file has not been saved yet.

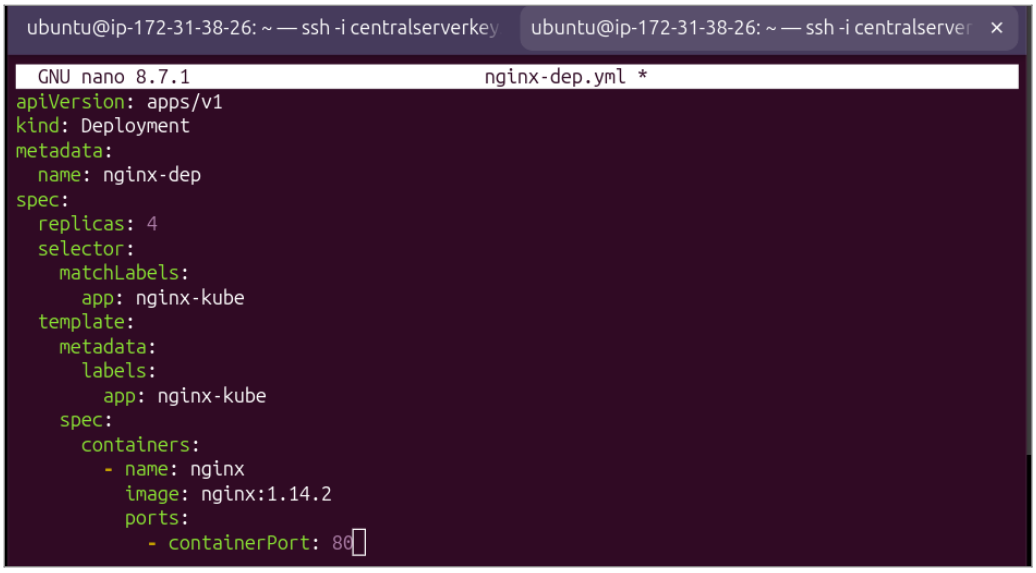


Figure 22: *Deployment manifest nginx-dep.yml in the nano editor*

8.2 Creating the Service Manifest

I then write the Service manifest `nginx-svc.yml`, which exposes the four Nginx pods through a single service.

Key fields in the file

- `kind: Service` and `name: nginx-svc` — the type and name of the resource.
- `type: LoadBalancer` — requests an external load-balanced service.
- `port: 80` and `targetPort: 80` — the service listens on port 80 and sends traffic to port 80 of the pods.
- `selector` — `app: nginx-kube`; the service sends traffic to pods with this label, which are the pods created by `nginx-dep`.

Observation

The editor shows the complete `nginx-svc.yml` manifest. The label in the selector matches the label used in the deployment manifest.

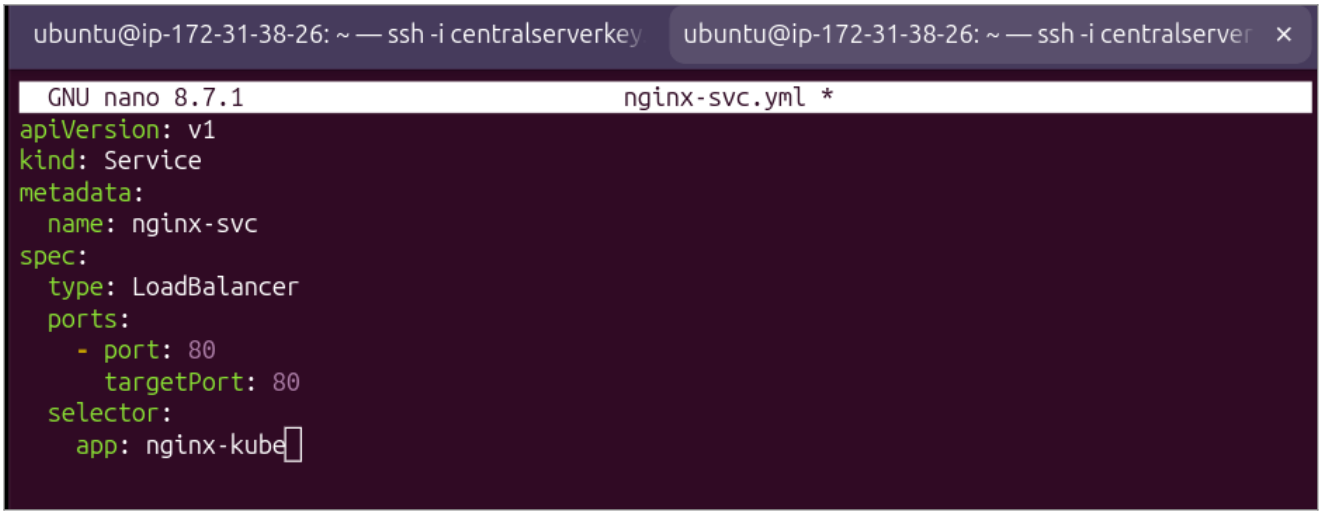


Figure 23: *Service manifest nginx-svc.yml in the nano editor*

8.3 Applying the Manifests

After saving both files, I apply them to the cluster so Kubernetes creates the resources they describe.

Commands used

```
nano nginx-dep.yml
nano nginx-svc.yml
kubectl apply -f nginx-dep.yml
kubectl apply -f nginx-svc.yml
```

Command explanation

- `nano nginx-dep.yml` and `nano nginx-svc.yml` — open the two manifest files in the nano editor.
- `kubectl apply -f nginx-dep.yml` — creates or updates the resources defined in the file; `-f` gives the file name.
- `kubectl apply -f nginx-svc.yml` — does the same for the service file.

Result

Kubernetes responds with `deployment.apps/nginx-dep created` and `service/nginx-svc created`, so both resources were created from the manifests.

```
ubuntu@ip-172-31-38-26:~$ nano nginx-dep.yml
ubuntu@ip-172-31-38-26:~$ nano nginx-svc.yml
ubuntu@ip-172-31-38-26:~$ kubectl apply -f nginx-dep.yml
deployment.apps/nginx-dep created
ubuntu@ip-172-31-38-26:~$ kubectl apply -f nginx-svc.yml
service/nginx-svc created
ubuntu@ip-172-31-38-26:~$
```

Figure 24: *Applying nginx-dep.yml and nginx-svc.yml with kubectl apply*

8.4 Verifying the YAML Deployment

I check the new deployment, service and pods. The `-l` option filters pods by label, so only the pods created by this deployment are listed.

Commands used

```
kubectl get deployment nginx-dep
kubectl get service nginx-svc
kubectl get pods -l app=nginx-kube
```

Command explanation

- `kubectl get deployment nginx-dep` → shows the deployment status.
- `kubectl get service nginx-svc` → shows the service details.
- `kubectl get pods -l app=nginx-kube` → `-l` selects pods that have the label `app=nginx-kube`.

Result

The deployment `nginx-dep` shows `4/4` ready. The service `nginx-svc` is of type `LoadBalancer` with cluster IP `10.109.177.192`, ports `80:30405/TCP` and an external IP of `<pending>`, which means no external address has been assigned, so the application is reached through port forwarding in the next step. Four pods named `nginx-dep-5fd6b8bd65-...` are `1/1 Running` with 0 restarts, matching the four replicas in the manifest.

```
ubuntu@ip-172-31-38-26:~$ kubectl get deployment nginx-dep
NAME        READY   UP-TO-DATE   AVAILABLE   AGE
nginx-dep   4/4     4            4           52s
ubuntu@ip-172-31-38-26:~$ kubectl get service nginx-svc
NAME        TYPE           CLUSTER-IP       EXTERNAL-IP   PORT(S)        AGE
nginx-svc   LoadBalancer   10.109.177.192   <pending>     80:30405/TCP   52s
ubuntu@ip-172-31-38-26:~$ kubectl get pods -l app=nginx-kube
NAME                         READY   STATUS    RESTARTS   AGE
nginx-dep-5fd6b8bd65-4cw4d   1/1     Running   0          76s
nginx-dep-5fd6b8bd65-bmb9g   1/1     Running   0          76s
nginx-dep-5fd6b8bd65-m98rl   1/1     Running   0          76s
nginx-dep-5fd6b8bd65-z5q95   1/1     Running   0          76s
ubuntu@ip-172-31-38-26:~$
```

Figure 25: *Verifying nginx-dep, nginx-svc and the four running pods*

8.5 Accessing the YAML-Deployed Nginx in the Browser

To open the new service from a browser, I forward server port 8080 to port 80 of the `nginx-svc` service.

Command used

```
kubectl port-forward service/nginx-svc 8080:80 --address=0.0.0.0
```

Command explanation

- `service/nginx-svc` — the service to forward to.
- `8080:80` — server port 8080 is forwarded to port 80 of the service.
- `--address=0.0.0.0` — listens on all network interfaces so the page can be opened through the public IP.

Result

The terminal prints `Forwarding from 0.0.0.0:8080 -> 80` and keeps running.

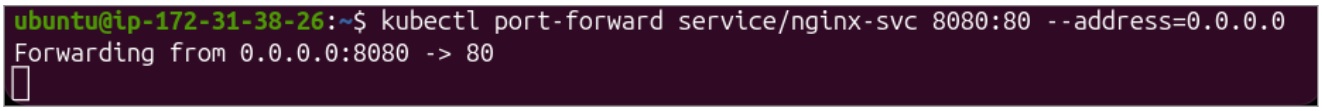


Figure 26: *Port-forwarding the nginx-svc service to port 8080*

I open the EC2 public IP address on port 8080 in the browser.

Address opened in the browser

```
16.171.154.62:8080
```

Address explanation

- `16.171.154.62:8080` — public IP of the EC2 instance and the forwarded port for `nginx-svc`.

Observation

The default “Welcome to nginx!” page is displayed, confirming that the application deployed from the YAML manifests is reachable.

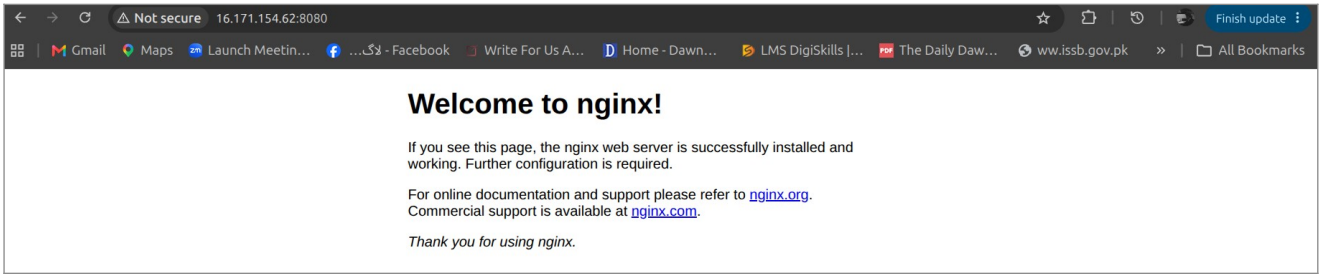


Figure 27: *Nginx welcome page opened at 16.171.154.62:8080*

8.6 Observing Kubernetes Self-Healing

Because the port forward occupies the first terminal, I open a second SSH session to the same server. The login banner of the new session is shown, followed by a command that lists the pods of the YAML deployment. The `-l` option selects pods by label.

Command used

```
kubectl get pods -l app=nginx-kube
```

Command explanation

- `kubectl get pods -l app=nginx-kube` — lists only the pods that carry the label `app=nginx-kube`.

Result

The login banner shows the server's IPv4 address `172.31.38.26`. The command lists four pods (`4cw4d`, `bmb9g`, `m98rl` and `z5q95` at the end of their names), all `1/1 Running` with 0 restarts and an age of about 5 minutes.

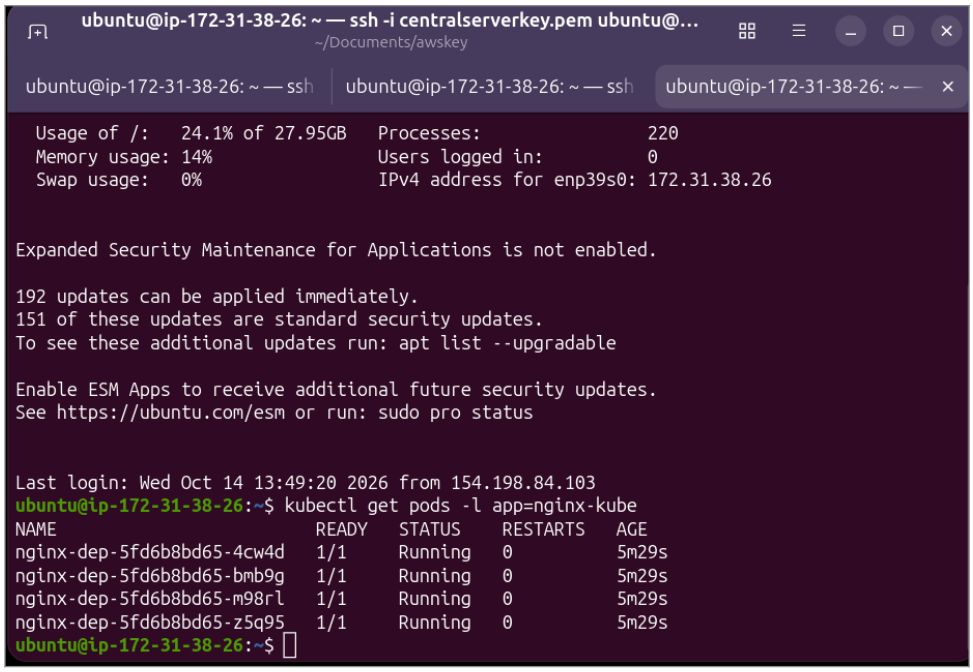


Figure 28: *Second SSH session listing the four nginx-dep pods*

Next, I test the self-healing behaviour of Kubernetes. One of the four pods is deleted manually, and the pods and deployment are listed again to see whether Kubernetes restores the desired number of replicas.

Commands used

```
kubectl delete pod nginx-dep-5fd6b8bd65-4cw4d
kubectl get pods -l app=nginx-kube
kubectl get deployment nginx-dep
```

Command explanation

- `kubectl delete pod nginx-dep-5fd6b8bd65-4cw4d` → deletes the pod with this name.
- `kubectl get pods -l app=nginx-kube` → lists the pods of the deployment again.
- `kubectl get deployment nginx-dep` → shows how many replicas are ready.

Result

Kubernetes confirms `pod "nginx-dep-5fd6b8bd65-4cw4d" deleted from default namespace`. The new pod list still contains four pods: the deleted pod is gone and a new pod, `nginx-dep-5fd6b8bd65-w6jw8`, appears with an age of only 11 seconds. The deployment still shows `4/4` ready. This demonstrates that the Deployment automatically creates a replacement pod to maintain the four replicas required by the manifest.

```
ubuntu@ip-172-31-38-26:~$ kubectl delete pod nginx-dep-5fd6b8bd65-4cw4d
pod "nginx-dep-5fd6b8bd65-4cw4d" deleted from default namespace
ubuntu@ip-172-31-38-26:~$ kubectl get pods -l app=nginx-kube
NAME                         READY   STATUS    RESTARTS   AGE
nginx-dep-5fd6b8bd65-bmb9g   1/1     Running   0          7m59s
nginx-dep-5fd6b8bd65-m98rl   1/1     Running   0          7m59s
nginx-dep-5fd6b8bd65-w6jw8   1/1     Running   0          11s
nginx-dep-5fd6b8bd65-z5q95   1/1     Running   0          7m59s
ubuntu@ip-172-31-38-26:~$ kubectl get deployment nginx-dep
NAME        READY   UP-TO-DATE   AVAILABLE   AGE
nginx-dep   4/4     4            4           8m13s
ubuntu@ip-172-31-38-26:~$
```

Figure 29: *Deleting a pod and Kubernetes automatically creating a replacement*

9. Deploying WordPress with MySQL

This section deploys a two-tier application: a WordPress web front end and a MySQL database. Each tier gets a Deployment and a Service. In real-world setups, splitting the application and the database into separate workloads allows each part to be managed and scaled independently.

9.1 Creating the WordPress Manifest

I create the file `wordpress.yml` in nano. It contains two Kubernetes resources separated by a line of three dashes (`---`): a Service and a Deployment for the WordPress front end. The file is shown in two screenshots, first its upper part and then its lower part.

Key fields in the file

- **Service `wordpress-frontend`** – type `LoadBalancer`, port 80 forwarded to `targetPort` 80, selecting pods with `app: wordpress-frontend`.
- **Deployment `wordpress-frontend`** – `replicas: 1`, so a single WordPress pod is run.
- **Container `wordpress`** – uses the image `wordpress:6.2.1-apache` and `containerPort: 80`.
- **Environment variables (`env`)** – `WORDPRESS_DB_HOST` is set to `wordpress-mysql:3306`, the name and port of the MySQL service, so WordPress can find its database. `WORDPRESS_DB_USER`, `WORDPRESS_DB_PASSWORD` and `WORDPRESS_DB_NAME` provide the database login details and database name.

Observation

The first screenshot shows the Service and the beginning of the Deployment; the second shows the pod template with the container, the database environment variables and the container port.

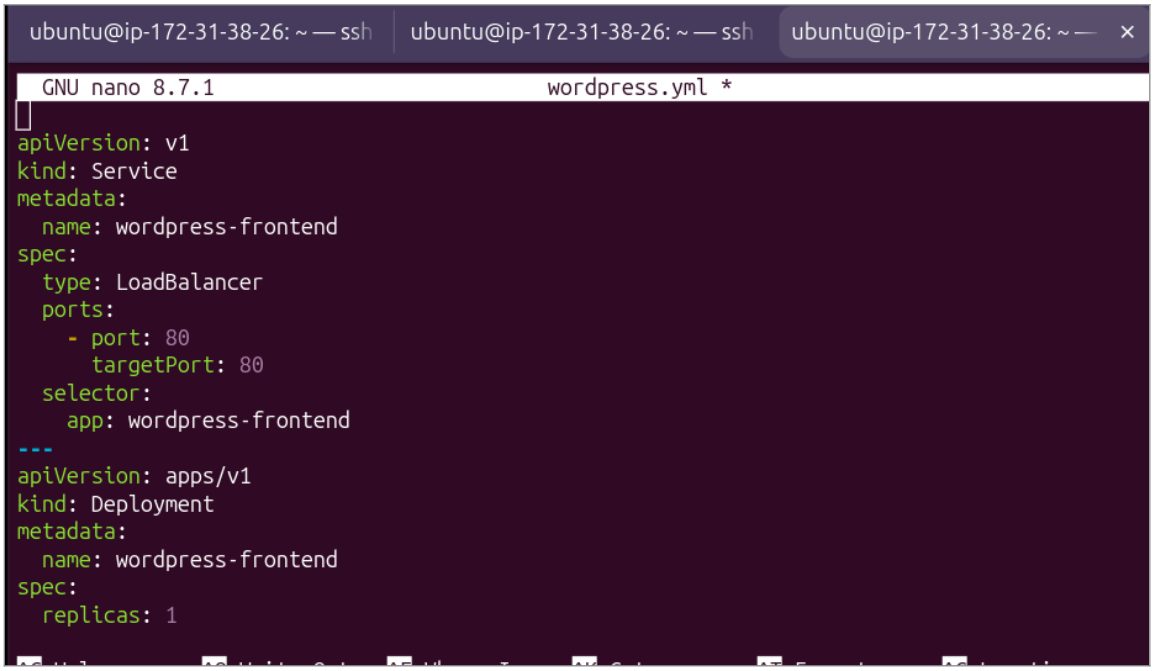


Figure 30: *wordpress.yml (upper part): wordpress-frontend Service and the start of the Deployment*

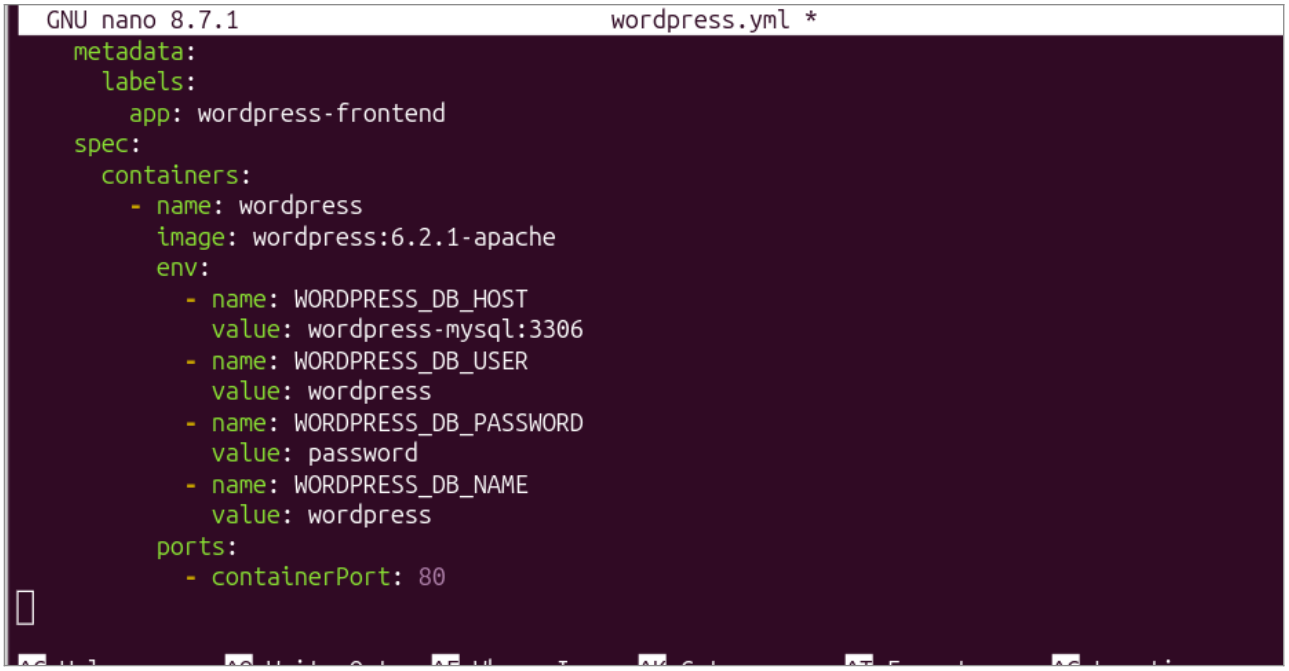


Figure 31: *wordpress.yml (lower part): container image, database environment variables and port*

9.2 Creating the MySQL Manifest

Next, I create `mysql.yml` for the database tier. The screenshot shows the upper part of the file; the remaining lines of the Deployment are below the visible area of the editor.

Key fields in the file

- **Service `wordpress-mysql`** → port 3306 forwarded to `targetPort` 3306, selecting pods with `app: wordpress-mysql`. No `type` is written, so Kubernetes uses the default internal type (ClusterIP), which is confirmed in the service list in Section 9.4.
- **Deployment `wordpress-mysql`** → `replicas: 1`, so a single database pod is run.

Observation

The editor shows `mysql.yml` with the service definition and the start of the deployment definition.

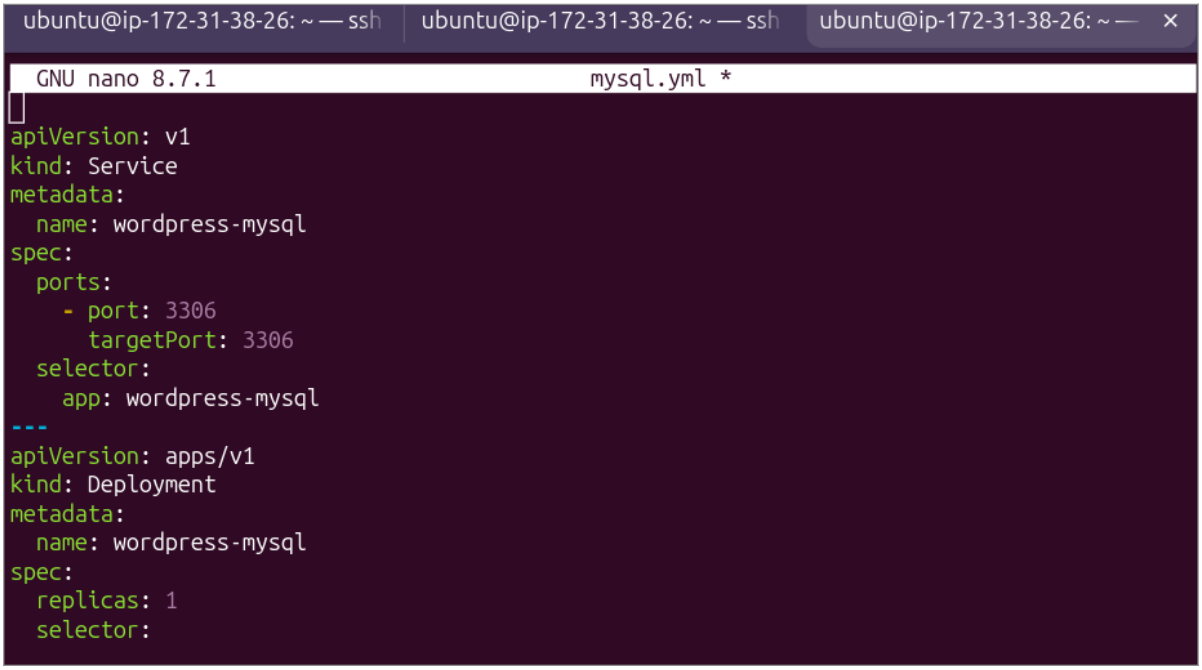


Figure 32: *mysql.yml in the nano editor: wordpress-mysql Service and Deployment*

9.3 Applying the WordPress and MySQL Manifests

I apply the database manifest first and then the WordPress manifest, so the database service exists before the WordPress front end starts.

Commands used

```
nano wordpress.yml
nano mysql.yml
kubectl apply -f mysql.yml
kubectl apply -f wordpress.yml
```

Command explanation

- `nano wordpress.yml` and `nano mysql.yml` → open the two manifest files in the nano editor.
- `kubectl apply -f mysql.yml` → creates the resources defined in `mysql.yml`.
- `kubectl apply -f wordpress.yml` → creates the resources defined in `wordpress.yml`.

Result

Kubernetes creates `service/wordpress-mysql` and `deployment.apps/wordpress-mysql`, and then `service/wordpress-frontend` and `deployment.apps/wordpress-frontend`.

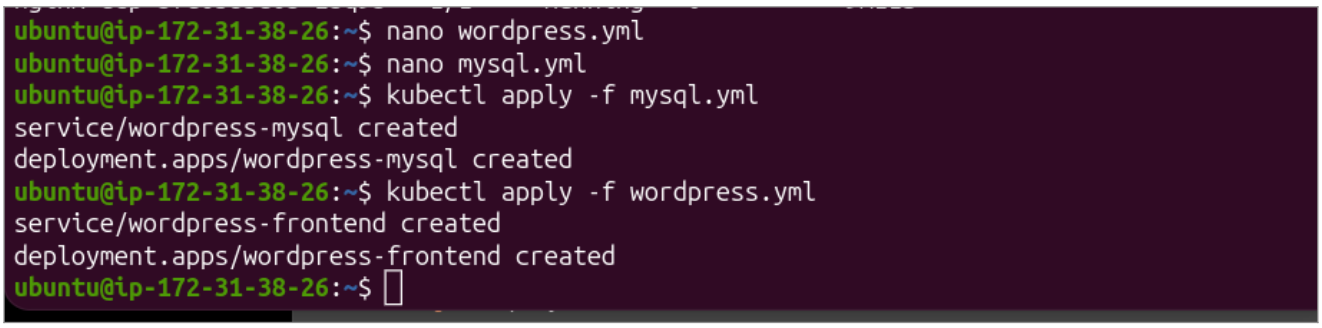


Figure 33: *Applying mysql.yml and wordpress.yml*

9.4 Verifying the WordPress and MySQL Resources

I list the deployments in the cluster to confirm that the new WordPress and MySQL deployments are ready.

Command used

```
kubectl get deployments
```

Command explanation

- `kubectl get deployments` — lists all deployments in the current namespace.

Result

Four deployments are listed: `nginx` (1/1), `nginx-dep` (4/4), `wordpress-frontend` (1/1) and `wordpress-mysql` (1/1). The two new deployments are ready and are only seconds old.



Figure 34: *Deployments listed, including wordpress-frontend and wordpress-mysql*

I then list the pods to confirm that the pods behind all deployments are running.

Command used

```
kubectl get pods
```

Command explanation

- `kubectl get pods` — lists all pods in the current namespace together with their status.

Result

All pods are `1/1 Running` with 0 restarts. They include the original nginx pod, the four `nginx-dep` pods (including the replacement pod `w6jw8` created earlier), `wordpress-frontend-6c76bd44-bwcx8` and `wordpress-mysql-5bc89547d7-tm76v`.

```
ubuntu@ip-172-31-38-26:~$ kubectl get pods
NAME                                READY   STATUS    RESTARTS   AGE
nginx-598cc96cd9-jhf89              1/1     Running   0          26m
nginx-dep-5fd6b8bd65-bmb9g          1/1     Running   0          13m
nginx-dep-5fd6b8bd65-m98rl          1/1     Running   0          13m
nginx-dep-5fd6b8bd65-w6jw8          1/1     Running   0          6m5s
nginx-dep-5fd6b8bd65-z5q95          1/1     Running   0          13m
wordpress-frontend-6c76bd44-bwcx8   1/1     Running   0          48s
wordpress-mysql-5bc89547d7-tm76v    1/1     Running   0          60s
ubuntu@ip-172-31-38-26:~$
```

Figure 35: *All pods running, including the WordPress and MySQL pods*

Finally, I list the services to check how the WordPress and MySQL tiers are exposed.

Command used

```
kubectl get services
```

Command explanation

- `kubectl get services` → lists all services with their type, internal IP address and ports.

Result

The list shows `wordpress-frontend` as a `LoadBalancer` service on `80:31766/TCP` (external IP `<pending>`) and `wordpress-mysql` as a `ClusterIP` service on `3306/TCP`, so the database is reachable only inside the cluster. The services `kubernetes`, `nginx` and `nginx-svc` from the earlier steps are also listed.

```
ubuntu@ip-172-31-38-26:~$ kubectl get services
NAME                TYPE           CLUSTER-IP       EXTERNAL-IP   PORT(S)        AGE
kubernetes          ClusterIP      10.96.0.1        <none>        443/TCP        31m
nginx               NodePort       10.111.198.135   <none>        80:30131/TCP   27m
nginx-svc           LoadBalancer   10.109.177.192   <pending>     80:30405/TCP   14m
wordpress-frontend  LoadBalancer   10.111.210.19    <pending>     80:31766/TCP   2m2s
wordpress-mysql     ClusterIP      10.111.191.104   <none>        3306/TCP       2m14s
ubuntu@ip-172-31-38-26:~$
```

Figure 36: *Services listed: wordpress-frontend (LoadBalancer) and wordpress-mysql (ClusterIP)*

9.5 Opening WordPress in the Browser

I list the deployments once more and then forward server port 8081 to the WordPress service so that the site can be opened from a browser.

Commands used

```
kubectl get deployments
kubectl port-forward service/wordpress-frontend 8081:80 --address=0.0.0.0
```

Command explanation

- `kubectl get deployments` → confirms the deployments are still ready.
- `kubectl port-forward service/wordpress-frontend 8081:80` → forwards server port 8081 to port 80 of the `wordpress-frontend` service.
- `--address=0.0.0.0` → listens on all network interfaces so the site is reachable via the public IP.

Result

All four deployments are ready (`1/1`, `4/4`, `1/1`, `1/1`). The terminal prints `Forwarding from 0.0.0.0:8081 -> 80` and keeps running.

```
ubuntu@ip-172-31-38-26:~$ kubectl get deployments
NAME                READY   UP-TO-DATE   AVAILABLE   AGE
nginx               1/1     1            1           30m
nginx-dep           4/4     4            4           18m
wordpress-frontend  1/1     1            1           5m11s
wordpress-mysql     1/1     1            1           5m23s
ubuntu@ip-172-31-38-26:~$ kubectl port-forward service/wordpress-frontend 8081:80 --address=
0.0.0.0
Forwarding from 0.0.0.0:8081 -> 80
```

Figure 37: *Port-forwarding the wordpress-frontend service to port 8081*

I open the WordPress installer through the forwarded port. The first page asks which language to use for the installation.

Address opened in the browser

```
16.171.154.62:8081/wp-admin/install.php
```

Address explanation

- `16.171.154.62:8081` → public IP and forwarded port of the WordPress service.
- `/wp-admin/install.php` → the WordPress installation page.

Observation

The WordPress installer loads and shows the language list with English (United States) selected. This shows that the WordPress pod is running and that it can serve pages.

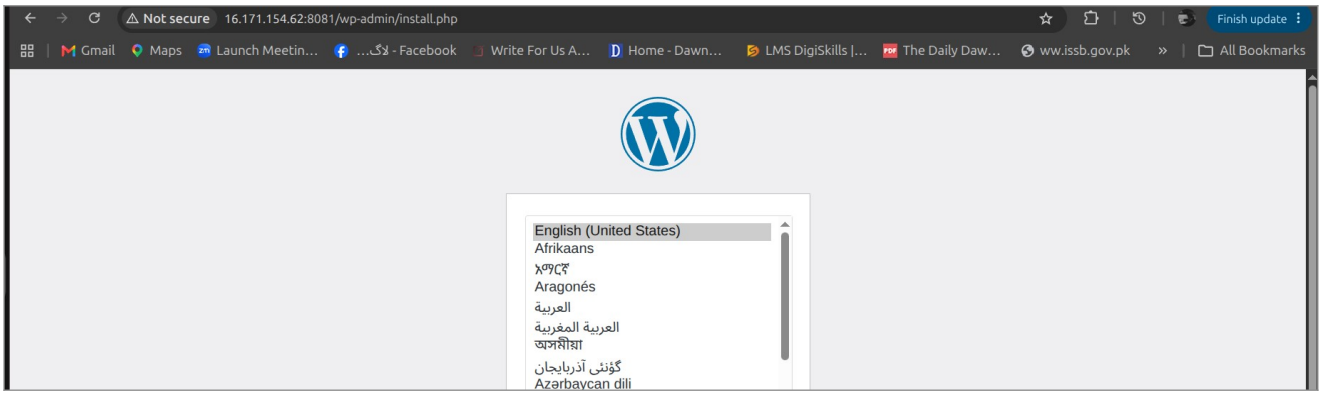


Figure 38: *WordPress installer language selection page*

9.6 Completing the WordPress Installation

After the language is chosen, WordPress shows its “Welcome” page. The site information is entered on this page: the site title, an administrator username, a password and an email address.

Fields on the page

- **Site Title**, **Username**, **Password** and **Your Email** → the details required to create the first administrator account.

- WordPress suggests a generated password and rates it as “Strong”.

Observation

The “Information needed” form is displayed with its fields still empty and a generated password suggestion.

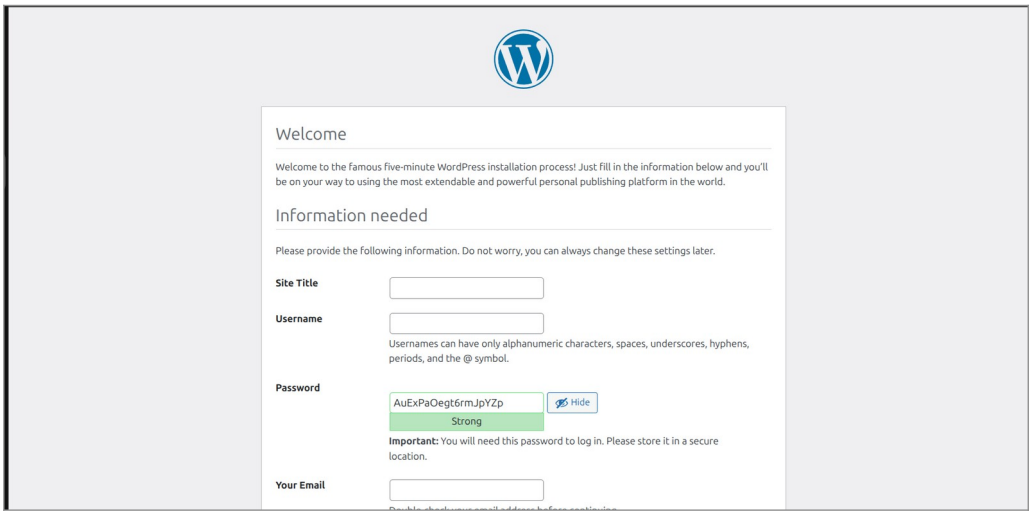


Figure 39: *WordPress “Welcome” page with the installation form*

I fill in the form. The site title is “Kubernetes Lab Website” and the username is `wpadmin_lab`. A different password is entered, which WordPress rates as weak, so the option to confirm its use is ticked. The search-engine visibility option is left unticked.

Fields on the page

- **Password strength** — shown as “Weak”, so WordPress requires “Confirm use of weak password” to be ticked.
- **Discourage search engines from indexing this site** — left unticked.

Observation

The form contains the site title, the username, a masked password with a “Weak” rating, the confirmation checkbox ticked and an email address.

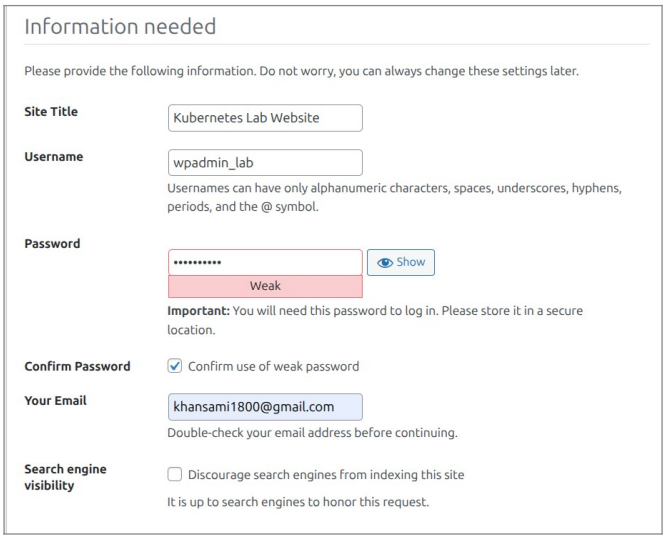


Figure 40: *Installation form completed with site title, username and password*

I submit the form to finish the installation.

Observation

The address changes to `install.php?step=2` and WordPress shows “Success! WordPress has been installed.” together with the username `wpadmin_lab` and a Log In button. The WordPress and MySQL pods are therefore working together: WordPress could write its installation data to the MySQL database.

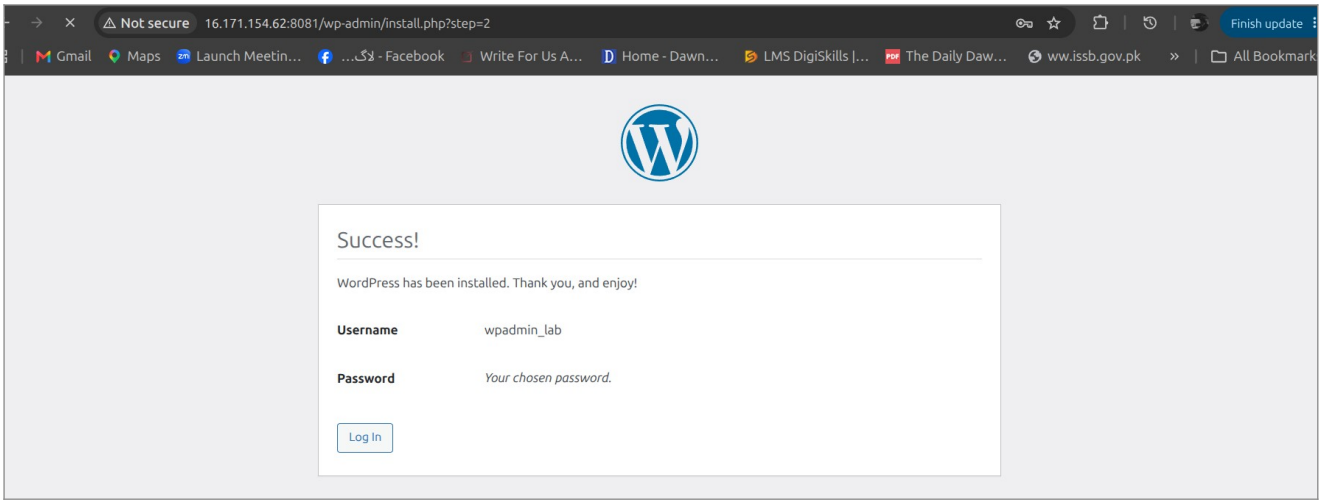


Figure 41: *WordPress installation completed successfully*

I log in with the new administrator account and reach the WordPress dashboard.

Address opened in the browser

```
16.171.154.62:8081/wp-admin/
```

Address explanation

- `/wp-admin/` → the WordPress administration area, opened after logging in.

Observation

The dashboard of “Kubernetes Lab Website” opens with the greeting “Howdy, wpadmin_lab” and the “Welcome to WordPress!” panel. A notice offers a newer WordPress version; the site runs version 6.2.1, which matches the image `wordpress:6.2.1-apache` used in the manifest.

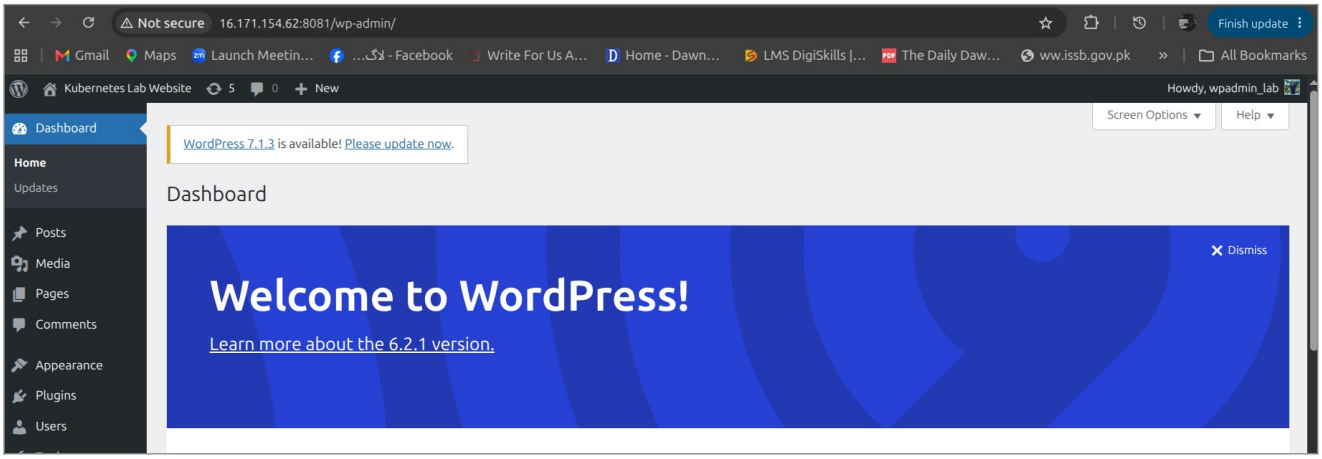


Figure 42: *WordPress dashboard after logging in*

10. Deploying the To-Do List Application

The last application is a to-do list web application, deployed with a Deployment of three replicas and a NodePort service. Using a separate file for each resource follows common Kubernetes practice.

10.1 Creating the To-Do Deployment Manifest

I create `deployment.yml` in nano. The file defines the Deployment that runs the to-do application. The screenshot shows the upper part of the file; the lines after the container name are below the visible area of the editor.

Key fields in the file

- `kind: Deployment` and `name: todo-deployment` — the type and name of the resource, with the label `app: todo-app`.
- `replicas: 3` — three identical pods must be kept running.
- `selector.matchLabels` and `template.metadata.labels` — both use `app: todo-app`, so the deployment manages the pods it creates.
- `containers` — the first container is named `todo-app`.

Observation

The editor shows `deployment.yml` with the deployment definition for `todo-deployment`.

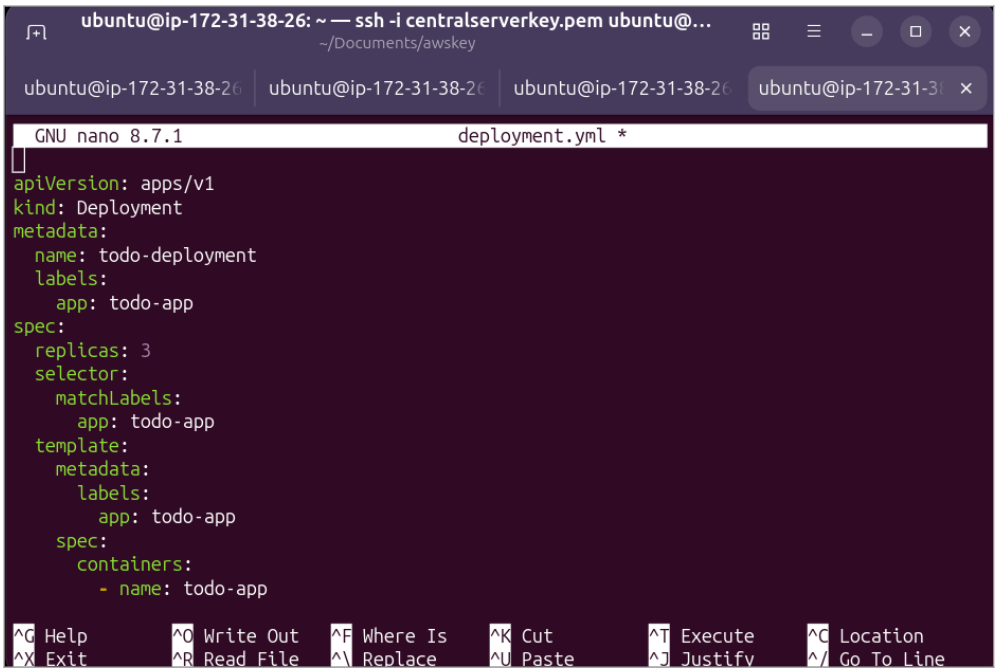


Figure 43: *deployment.yml in nano: todo-deployment with 3 replicas*

10.2 Creating the To-Do Service Manifest

I create `service.yml`, which exposes the to-do pods through a NodePort service.

Key fields in the file

- `kind: Service` and `name: todo-service` — the type and name of the resource.
- `type: NodePort` — makes the service available on a port of the node.

- `selector` → `app: todo-app`; selects the pods created by `todo-deployment`.
- `port: 80` → the service port.
- `targetPort: 8000` → the container port that receives the traffic.
- `nodePort: 30007` → the fixed node port assigned to the service.

Observation

The editor shows the complete `service.yml` manifest for `todo-service`.

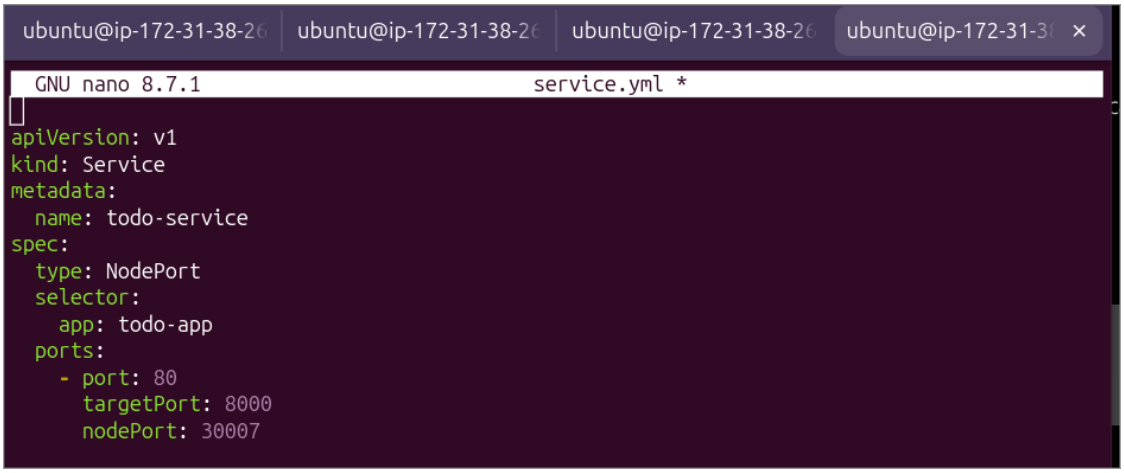


Figure 44: *service.yml in nano: todo-service of type NodePort*

10.3 Applying the To-Do Manifests

I open both files in nano and then apply them to the cluster.

Commands used

```
nano deployment.yml
nano service.yml
```

Command explanation

- `nano deployment.yml` and `nano service.yml` → open the two manifest files in the nano editor.

Result

The terminal shows the two editor commands; the files were written in the previous steps.

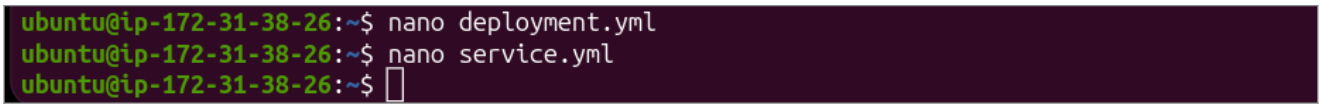


Figure 45: *Opening deployment.yml and service.yml in nano*

The two manifests are applied to the cluster.

Commands used

```
kubectl apply -f deployment.yml
kubectl apply -f service.yml
```

Command explanation

- `kubectl apply -f deployment.yml` → creates the to-do deployment.

- `kubectl apply -f service.yml` → creates the to-do service.

Result

Kubernetes replies `deployment.apps/todo-deployment created` and `service/todo-service created`.

```
ubuntu@ip-172-31-38-26:~$ kubectl apply -f deployment.yml
deployment.apps/todo-deployment created
ubuntu@ip-172-31-38-26:~$ kubectl apply -f service.yml
service/todo-service created
ubuntu@ip-172-31-38-26:~$
```

Figure 46: *Applying deployment.yml and service.yml*

10.4 Verifying the To-Do Application

I check the deployment, its pods and the service.

Commands used

```
kubectl get deployment todo-deployment
kubectl get pods -l app=todo-app
kubectl get service todo-service
```

Command explanation

- `kubectl get deployment todo-deployment` → shows the deployment status.
- `kubectl get pods -l app=todo-app` → lists the pods with the label `app=todo-app`.
- `kubectl get service todo-service` → shows the service details.

Result

The deployment shows `3/3` ready. Three `todo-deployment-79ddc7dc7-...` pods are `1/1 Running` with 0 restarts. The service `todo-service` is of type `NodePort` with cluster IP `10.105.145.167` and ports `80:30007/TCP`, which matches the `nodePort` in the manifest.

```
ubuntu@ip-172-31-38-26:~$ kubectl get deployment todo-deployment
NAME              READY   UP-TO-DATE   AVAILABLE   AGE
todo-deployment   3/3     3            3           55s
ubuntu@ip-172-31-38-26:~$ kubectl get pods -l app=todo-app
NAME                               READY   STATUS    RESTARTS   AGE
todo-deployment-79ddc7dc7-9nmcq    1/1     Running   0          65s
todo-deployment-79ddc7dc7-nh5r6    1/1     Running   0          65s
todo-deployment-79ddc7dc7-rlcj4    1/1     Running   0          65s
ubuntu@ip-172-31-38-26:~$ kubectl get service todo-service
NAME           TYPE       CLUSTER-IP       EXTERNAL-IP   PORT(S)        AGE
todo-service   NodePort   10.105.145.167   <none>        80:30007/TCP   67s
ubuntu@ip-172-31-38-26:~$
```

Figure 47: *To-do deployment, three pods and NodePort service verified*

10.5 Using the To-Do Application in the Browser

The to-do application is opened through the EC2 public IP on port 8083. This port is forwarded to the `todo-service` service; the port-forward terminal for it is shown in Section 11.

Address opened in the browser

```
16.171.154.62:8083/todos/
```

Address explanation

- `16.171.154.62:8083` — public IP and the forwarded port for the to-do service.
- `/todos/` — the path of the to-do page.

Observation

The page titled “Todo List - Modified once again” loads with an input box (showing “Do laundry”) and an Add button, which shows the application is running and reachable.

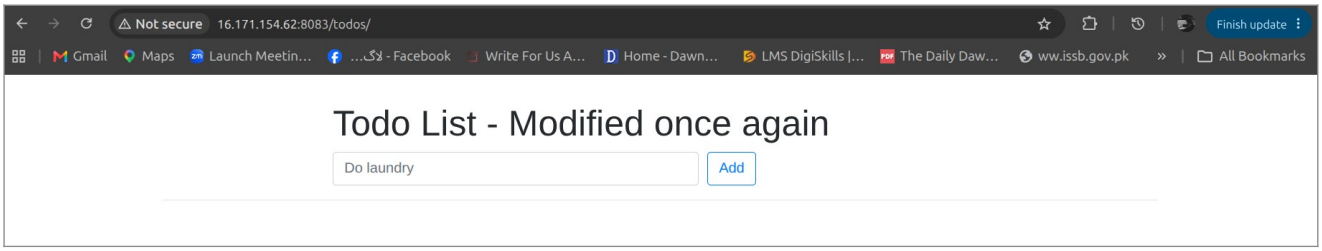


Figure 48: *To-do application opened at 16.171.154.62:8083/todos/*

I use the application by adding tasks to the list.

Observation

The list now contains three tasks: “Turn off EC2 Instance”, “Submit German Assignment” and “Upload Lab Task”. The last task is ticked and shown struck through, meaning it is marked as completed.

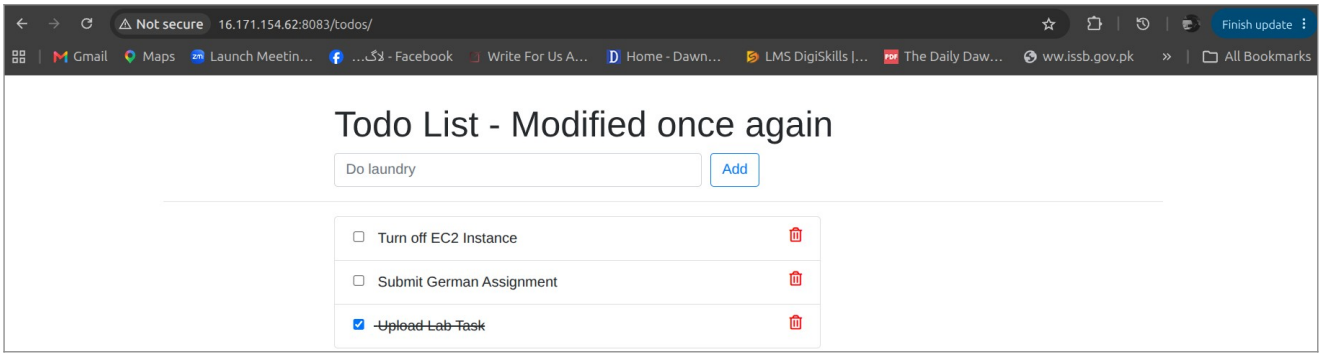


Figure 49: *To-do application showing three tasks, one marked completed*

11. Port-Forward Sessions for All Applications

Each application is made reachable on its own port by a separate `kubectl port-forward` command. Every command keeps running in its own SSH terminal, and the `Handling connection for ...` lines appear whenever the browser connects. The screenshots below show these terminals.

11.1 Nginx on Port 8082

This terminal belongs to the first Nginx deployment created with kubectl commands (Section 7.3).

Command used

```
kubectl port-forward service/nginx 8082:80 --address=0.0.0.0
```

Command explanation

- The command forwards server port 8082 to port 80 of the `nginx` service, listening on all interfaces.

Result

After `Forwarding from 0.0.0.0:8082 -> 80`, two `Handling connection for 8082` lines appear, showing that the browser has connected to the application through this port.

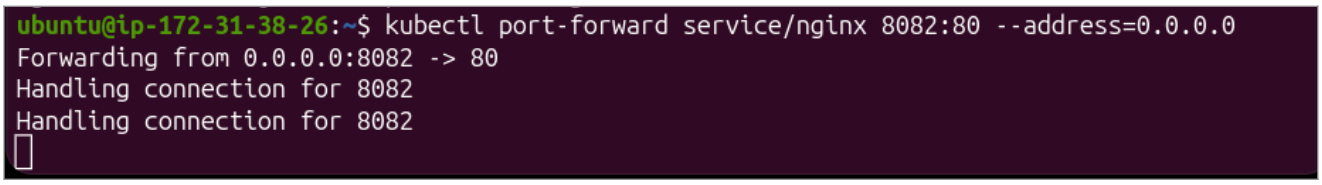


Figure 50: *Port-forward terminal for the nginx service on port 8082*

11.2 To-Do Application on Port 8083

This terminal serves the to-do application. The service port 80 is forwarded to the container port 8000, as defined by `targetPort` in `service.yml`.

Command used

```
kubectl port-forward service/todo-service 8083:80 --address=0.0.0.0
```

Command explanation

- `service/todo-service` — the service to forward to.
- `8083:80` — server port 8083 is forwarded to port 80 of the service.
- `--address=0.0.0.0` — listens on all network interfaces.

Result

The terminal prints `Forwarding from 0.0.0.0:8083 -> 8000`, so traffic arriving on port 8083 is delivered to port 8000 of the to-do pods.

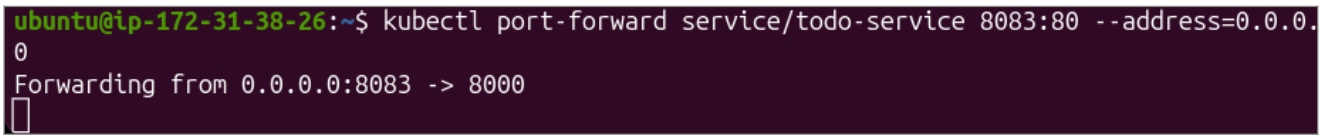


Figure 51: *Starting the port forward for the to-do service on port 8083*

11.3 Nginx (YAML) on Port 8080

This terminal serves the Nginx application deployed from the YAML manifests (Section 8.5).

Command used

```
kubectl port-forward service/nginx-svc 8080:80 --address=0.0.0.0
```

Command explanation

- The command forwards server port 8080 to port 80 of the `nginx-svc` service.

Result

After `Forwarding from 0.0.0.0:8080 -> 80`, three `Handling connection for 8080` lines show that the application was opened from the browser.

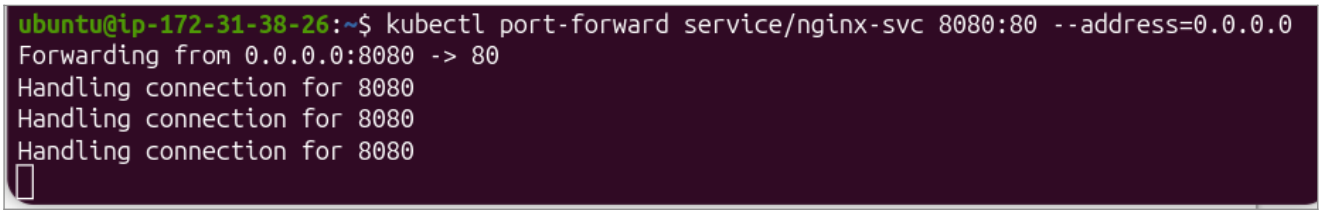


Figure 52: *Port-forward terminal for the nginx-svc service on port 8080*

11.4 WordPress on Port 8081

This terminal serves the WordPress site (Section 9.5).

Command used

```
kubectl port-forward service/wordpress-frontend 8081:80 --address=0.0.0.0
```

Command explanation

- The command forwards server port 8081 to port 80 of the `wordpress-frontend` service.

Result

After `Forwarding from 0.0.0.0:8081 -> 80`, several `Handling connection for 8081` lines appear. WordPress loads many pages and files, so it creates more connections than the other applications.

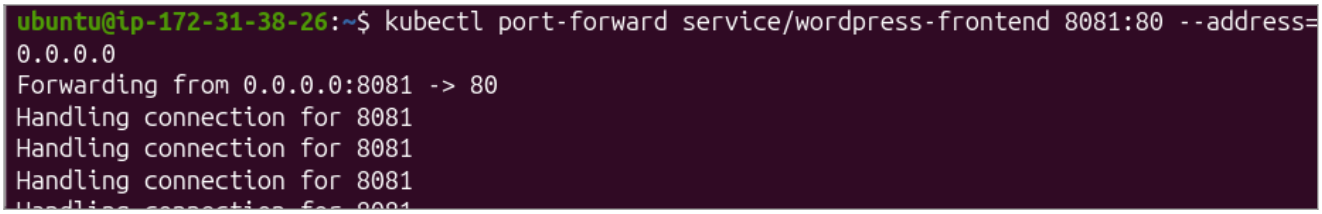


Figure 53: *Port-forward terminal for the wordpress-frontend service on port 8081*

11.5 To-Do Application with Active Connections

The to-do port-forward command is shown again, this time after the application has been used in the browser.

Command used

```
kubectl port-forward service/todo-service 8083:80 --address=0.0.0.0
```

Command explanation

- The command is the same as in Section 11.2.

Result

After `Forwarding from 0.0.0.0:8083 -> 8000`, several `Handling connection for 8083` lines appear, showing that browser requests are being received by the to-do application.

```
ubuntu@ip-172-31-38-26:~$ kubectl port-forward service/todo-service 8083:80 --address=0.0.0.
0
Forwarding from 0.0.0.0:8083 -> 8000
Handling connection for 8083
Handling connection for 8083
Handling connection for 8083
```

Figure 54: *Port-forward terminal for the to-do service with active connections*

11.6 All Applications Running Together

With all port-forward terminals active, the four applications are opened in separate browser windows, each through the same EC2 public IP and a different port.

Windows shown

- `16.171.154.62:8082` → "Welcome to nginx!" (Nginx created with kubectl commands).
- `16.171.154.62:8083/todos/` → the to-do list application.
- `16.171.154.62:8080` → "Welcome to nginx!" (Nginx created from YAML manifests).
- `16.171.154.62:8081/wp-a...` → the WordPress dashboard.

Observation

All four applications are running in the same Kubernetes cluster and can be opened at the same time, each on its own port.

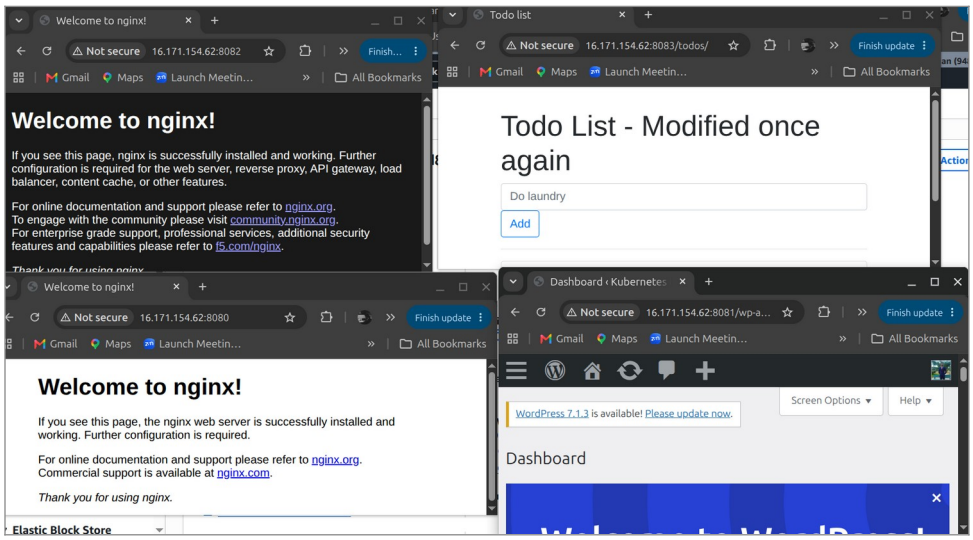


Figure 55: *Nginx (8082 and 8080), to-do list (8083) and WordPress dashboard (8081) open together*

12. Final Verification of the Cluster

At the end of the lab, I review the whole cluster to confirm that every workload is still running.

12.1 Nodes and Deployments

I check the node and then list all deployments.

Commands used

```
kubectl get nodes
kubectl get deployments
```

Command explanation

- `kubectl get nodes` – shows the status of the cluster node.
- `kubectl get deployments` – lists all deployments with their ready replicas.

Result

The node `minikube` is `Ready` with the role `control-plane` and version `v1.37.0`. The deployments shown are `nginx` (1/1), `nginx-dep` (4/4), `todo-deployment` (3/3), `wordpress-frontend` (1/1) and `wordpress-mysql` (1/1). The screenshot is cut at the bottom of the terminal after the `wordpress-mysql` line.

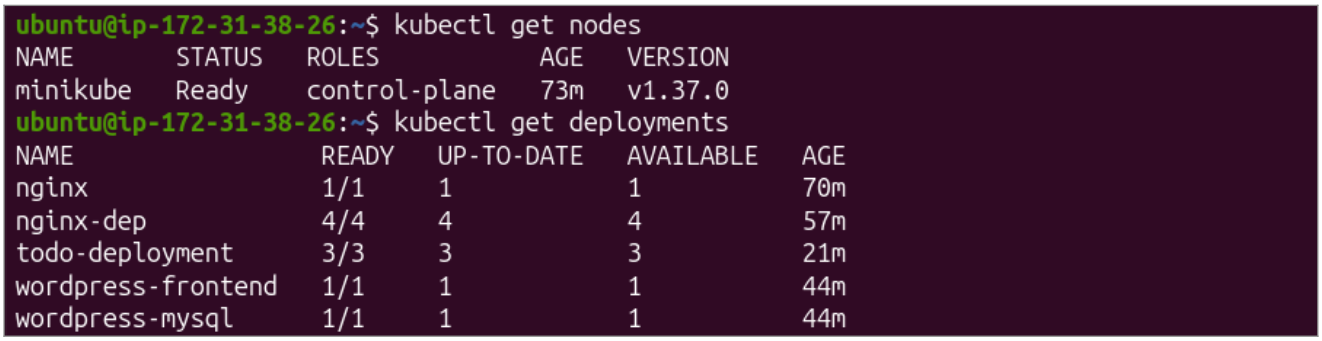


Figure 56: *Final check of the node and all deployments*

12.2 Services and Pods

I list all services and all pods for a final overview of the cluster.

Commands used

```
kubectl get services
kubectl get pods
```

Command explanation

- `kubectl get services` – lists all services with their type, IP address and ports.
- `kubectl get pods` – lists all pods with their status.

Result

Six services are listed: `kubernetes` (ClusterIP), `nginx` (NodePort, `80:30131/TCP`), `nginx-svc` (LoadBalancer, `80:30405/TCP`), `todo-service` (NodePort, `80:30007/TCP`), `wordpress-frontend` (LoadBalancer, `80:31766/TCP`) and `wordpress-mysql` (ClusterIP, `3306/TCP`). Ten pods are listed: one nginx pod, four `nginx-dep` pods, three `todo-deployment` pods, one `wordpress-frontend` pod and one `wordpress-mysql` pod. All are `1/1 Running` with 0 restarts.

```
ubuntu@ip-172-31-38-26:~$ kubectl get services
NAME                TYPE           CLUSTER-IP       EXTERNAL-IP   PORT(S)        AGE
kubernetes          ClusterIP      10.96.0.1        <none>        443/TCP        74m
nginx               NodePort       10.111.198.135   <none>        80:30131/TCP   70m
nginx-svc           LoadBalancer   10.109.177.192   <pending>     80:30405/TCP   57m
todo-service        NodePort       10.105.145.167   <none>        80:30007/TCP   21m
wordpress-frontend  LoadBalancer   10.111.210.19    <pending>     80:31766/TCP   44m
wordpress-mysql     ClusterIP      10.111.191.104   <none>        3306/TCP       45m
ubuntu@ip-172-31-38-26:~$ kubectl get pods
NAME                                READY   STATUS    RESTARTS   AGE
nginx-598cc96cd9-jhf89              1/1     Running   0          70m
nginx-dep-5fd6b8bd65-bmb9g          1/1     Running   0          58m
nginx-dep-5fd6b8bd65-m98rl          1/1     Running   0          58m
nginx-dep-5fd6b8bd65-w6jw8          1/1     Running   0          50m
nginx-dep-5fd6b8bd65-z5q95          1/1     Running   0          58m
todo-deployment-79ddc7dc7-9nmcq     1/1     Running   0          21m
todo-deployment-79ddc7dc7-nh5r6     1/1     Running   0          21m
todo-deployment-79ddc7dc7-rlcj4     1/1     Running   0          21m
wordpress-frontend-6c76bd44-bwcx8   1/1     Running   0          45m
wordpress-mysql-5bc89547d7-tm76v    1/1     Running   0          45m
ubuntu@ip-172-31-38-26:~$
```

Figure 57: *Final check of all services and pods*

12.3 Self-Healing Test: Deleting a Pod Again

To demonstrate self-healing once more, I first list the four pods of the `nginx-dep` deployment.

Command used

```
kubectl get pods -l app=nginx-kube
```

Command explanation

- `kubectl get pods -l app=nginx-kube` → lists the pods with the label `app=nginx-kube`.

Result

Four pods are running: `bmb9g`, `m98rl`, `w6jw8` and `z5q95` (name endings), each `1/1 Running` with 0 restarts.

```
ubuntu@ip-172-31-38-26:~$ kubectl get pods -l app=nginx-kube
NAME                         READY   STATUS    RESTARTS   AGE
nginx-dep-5fd6b8bd65-bmb9g   1/1     Running   0          61m
nginx-dep-5fd6b8bd65-m98rl   1/1     Running   0          61m
nginx-dep-5fd6b8bd65-w6jw8   1/1     Running   0          53m
nginx-dep-5fd6b8bd65-z5q95   1/1     Running   0          61m
ubuntu@ip-172-31-38-26:~$
```

Figure 58: *The four nginx-dep pods before the deletion test*

I delete one of these pods, `nginx-dep-5fd6b8bd65-bmb9g`.

Command used

```
kubectl delete pod nginx-dep-5fd6b8bd65-bmb9g
```

Command explanation

- `kubectl delete pod nginx-dep-5fd6b8bd65-bmb9g` → deletes the pod with this name.

Result

Kubernetes confirms `pod "nginx-dep-5fd6b8bd65-bmb9g" deleted from default namespace`.

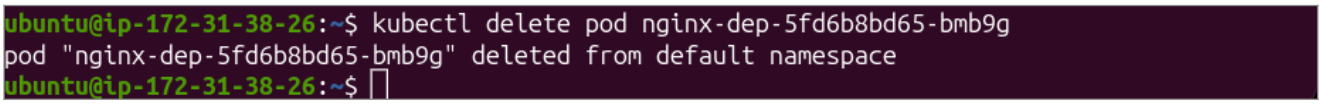


Figure 59: *Deleting the pod nginx-dep-5fd6b8bd65-bmb9g*

I watch the pod list to see how Kubernetes reacts to the deletion.

Command used

```
kubectl get pods -l app=nginx-kube -w
```

Command explanation

- `kubectl get pods -l app=nginx-kube` → lists the pods of the deployment.
- `-w` → "watch" mode; the command keeps running and prints any change to the pods.

Result

The deleted pod `bmb9g` is no longer listed. A new pod, `nginx-dep-5fd6b8bd65-qx9tf`, is running with an age of 27 seconds, and the other three pods are unchanged. There are again four running pods, which confirms that Kubernetes automatically replaced the deleted pod to keep the desired number of replicas.



Figure 60: *Watching Kubernetes create the replacement pod qx9tf*

PART 2: MULTINODE KUBERNETES CLUSTER IMPLEMENTATION

Part 1 of this report built a single-node Minikube cluster and deployed Nginx, WordPress with MySQL and a Django to-do application on it. Part 2 configures that existing one-node setup as a cluster with a second node, instead of rebuilding everything from scratch, so that the lab meets the requirement of a Kubernetes cluster with at least two nodes. A worker node, `minikube-m02`, is added to the control-plane node `minikube`. The three replicas of the to-do application are then moved to the worker node with a node selector, while Nginx, WordPress and MySQL stay on `minikube`.

Both nodes are Docker containers created by Minikube's Docker driver on the same EC2 instance. The result is a two-node logical cluster; it is not two separate EC2 instances or two physical machines. The screenshots supplied for Part 2 carry no captions or numbers of their own, so they are numbered Figure 61 to Figure 80 in the order in which they appear in the source screenshot document, continuing the numbering of Part 1.

13. Verifying the Existing Cluster Before Adding the Worker Node

Before a second node is added, I restart and check the existing cluster to confirm that it is healthy and that all workloads from Part 1 are available.

13.1 Restarting the Minikube Cluster

Purpose

The cluster from Part 1 is started again so that the existing node is up and can be used as the base of the multinode cluster.

Command used

```
minikube start --driver=docker
```

Command explanation

- `minikube start` → starts the existing Minikube cluster.
- `--driver=docker` → runs the node as a Docker container, the same driver used in Part 1.

Observed result

Minikube v1.39.0 reports that it is using the docker driver based on the existing profile, starts the primary control-plane node of the "minikube" cluster and restarts the existing Docker container for "minikube". It prepares Kubernetes v1.37.0 on containerd 2.3.4, verifies the Kubernetes components and enables the addons `storage-provisioner` and `default-storageclass`. A warning states that kubectl 1.35.9 may be incompatible with Kubernetes 1.37.0; it is only a warning, and the kubectl commands in the following figures work. The last line confirms that kubectl is configured to use the "minikube" cluster and the default namespace.



Figure 61: *Restarting the existing Minikube cluster with minikube start*

13.2 Checking the Cluster Status, Node and Deployments

Purpose

This check records the starting point: a healthy single-node cluster with all Part 1 deployments available.

Commands used

```
minikube status
kubectl get nodes -o wide
kubectl get deployments
```

Command explanation

- `minikube status` – shows the state of the Minikube node, its kubelet and its API server.
- `kubectl get nodes -o wide` – lists the nodes with extra columns such as internal IP, OS image and container runtime.
- `kubectl get deployments` – lists the deployments with their ready, up-to-date and available replica counts.

Observed result

Minikube reports the node `minikube` as type Control Plane with host, kubelet and apiserver Running and kubeconfig Configured. kubectl shows one node, `minikube`, which is Ready with the role control-plane, an age of 30 hours, version v1.37.0, internal IP 192.168.49.2 and container runtime containerd://2.3.4. The deployments are `nginx` (1/1), `nginx-dep` (4/4), `todo-deployment` (3/3), `wordpress-frontend` (1/1) and `wordpress-mysql` (1/1), all available. This confirms that the cluster has a single node and that all Part 1 workloads are running.

```
ubuntu@ip-172-31-38-26:~$ minikube status
minikube
type: Control Plane
host: Running
kubelet: Running
apiserver: Running
kubeconfig: Configured

ubuntu@ip-172-31-38-26:~$ kubectl get nodes -o wide
NAME       STATUS   ROLES           AGE   VERSION   INTERNAL-IP    EXTERNAL-IP   OS-IMAGE
                    KERNEL-VERSION           CONTAINER-RUNTIME
minikube   Ready    control-plane   30h   v1.37.0   192.168.49.2   <none>        Debian GNU/
Linux 12 (bookworm)   7.0.0-1006-aws (amd64)   containerd://2.3.4
ubuntu@ip-172-31-38-26:~$ kubectl get deployments
NAME                READY   UP-TO-DATE   AVAILABLE   AGE
nginx               1/1     1            1           30h
nginx-dep           4/4     4            4           29h
todo-deployment     3/3     3            3           29h
wordpress-frontend  1/1     1            1           29h
wordpress-mysql     1/1     1            1           29h
ubuntu@ip-172-31-38-26:~$
```

Figure 62: *Status, node and deployments of the single-node cluster before the worker is added*

14. Adding the Worker Node

I now add a second node to the existing Minikube cluster and check that Minikube and Kubernetes both see it.

14.1 Adding the Worker Node

Purpose

A second node is needed so that the cluster has two nodes and workloads can be placed on a node other than the original one.

Command used

```
minikube node add --worker
```

Command explanation

- `minikube node add` – creates an additional node in the existing Minikube cluster.
- `--worker` – makes the new node a worker only, so it does not run a second control plane.

Observed result

Minikube adds node m02 to the cluster "minikube" as a worker, starts the worker node "minikube-m02" and pulls the base image v0.0.51. It creates a Docker container for the node with 2 CPUs and 3072 MB of memory, prepares Kubernetes v1.37.0 on containerd 2.3.4, verifies the Kubernetes components and ends with "Successfully added m02 to minikube!". The new node is a container on the same EC2 instance.

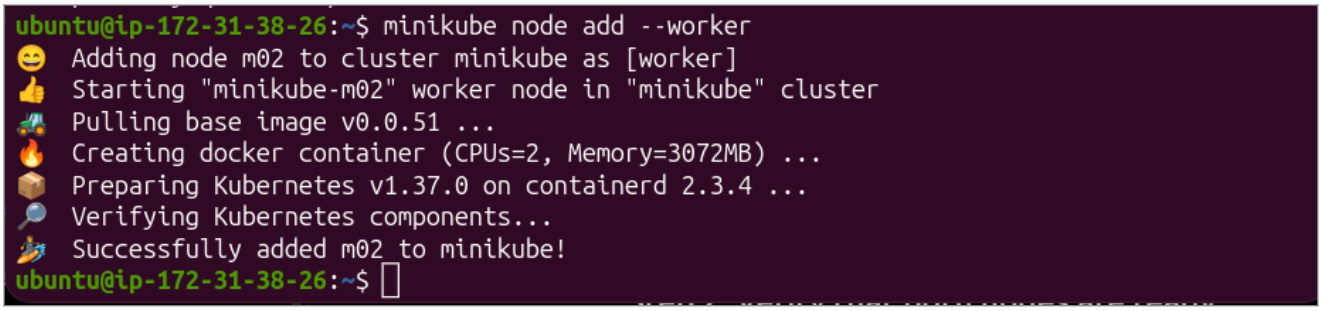


Figure 63: *Adding the worker node minikube-m02 with minikube node add --worker*

14.2 Listing the Minikube Nodes

Purpose

This check confirms from Minikube's side that the cluster now contains two nodes.

Command used

```
minikube node list
```

Command explanation

- `minikube node list` – prints each node of the Minikube cluster with its IP address.

Observed result

Two nodes are listed: `minikube` with the IP address 192.168.49.2 and `minikube-m02` with the IP address 192.168.49.3. These are internal addresses of the Minikube nodes, not the EC2 address 172.31.38.26 shown in the prompt.

```
ubuntu@ip-172-31-38-26:~$ minikube node list
minikube        192.168.49.2
minikube-m02    192.168.49.3
ubuntu@ip-172-31-38-26:~$
```

Figure 64: *The two nodes of the cluster shown by minikube node list*

14.3 Verifying the Nodes with kubectl

Purpose

The new node must also be registered in Kubernetes and be Ready before it can run pods.

Command used

```
kubectl get nodes -o wide
```

Command explanation

- `kubectl get nodes -o wide` – lists the Kubernetes nodes with their status, role, age, version, IP addresses, operating system and container runtime.

Observed result

`minikube` is Ready with the role control-plane (age 30h, IP 192.168.49.2). `minikube-m02` is also Ready, with age 4m36s, version v1.37.0 and IP 192.168.49.3. The ROLES column of `minikube-m02` shows <none> because no role label has been assigned to it yet. Both nodes use Debian GNU/Linux 12 (bookworm), kernel 7.0.0-1006-aws and containerd://2.3.4. The terminal in this capture is narrow, so the output wraps onto two lines per node; the same command is shown with a wider terminal in Figure 68.

```
ubuntu@ip-172-31-38-26:~$ kubectl get nodes -o wide
NAME           STATUS   ROLES           AGE     VERSION   INTERNAL-IP    EXTERNAL-IP   OS-IM
AGE                     KERNEL-VERSION           CONTAINER-RUNTIME
minikube       Ready    control-plane   30h     v1.37.0   192.168.49.2   <none>        Debia
n GNU/Linux 12 (bookworm)   7.0.0-1006-aws (amd64)   containerd://2.3.4
minikube-m02   Ready    <none>          4m36s   v1.37.0   192.168.49.3   <none>        Debia
n GNU/Linux 12 (bookworm)   7.0.0-1006-aws (amd64)   containerd://2.3.4
ubuntu@ip-172-31-38-26:~$
```

Figure 65: *Both nodes registered in Kubernetes and in the Ready state*

15. Reviewing the Existing Workloads

Before any workload is moved, I review the deployments and the placement of all pods.

15.1 Reviewing the Deployments

Purpose

This check confirms that the deployments are still available after the new node was added.

Command used

```
kubectl get deployments
```

Command explanation

- `kubectl get deployments` – lists the deployments with their READY, UP-TO-DATE, AVAILABLE and AGE values.

Observed result

The five deployments are unchanged: `nginx` 1/1, `nginx-dep` 4/4, `todo-deployment` 3/3, `wordpress-frontend` 1/1 and `wordpress-mysql` 1/1, each with all replicas up to date and available. Adding the worker node did not disturb them.

```
ubuntu@ip-172-31-38-26:~$ kubectl get deployments
NAME                READY   UP-TO-DATE   AVAILABLE   AGE
nginx               1/1     1            1           30h
nginx-dep           4/4     4            4           29h
todo-deployment     3/3     3            3           29h
wordpress-frontend  1/1     1            1           29h
wordpress-mysql     1/1     1            1           29h
```

Figure 66: *Deployments reviewed after the worker node was added*

15.2 Checking Pod Placement Before Scheduling

Purpose

I need to know on which node each pod is currently running before the to-do pods are moved.

Command used

```
kubectl get pods -o wide
```

Command explanation

- `kubectl get pods -o wide` – lists the pods with the extra columns IP, NODE, NOMINATED NODE and READINESS GATES; the NODE column shows where each pod runs.

Observed result

Ten pods are listed, all 1/1 Running and all on the node `minikube`: one `nginx` pod, four `nginx-dep` pods, three `todo-deployment` pods (name prefix 79ddc7dc7), one `wordpress-frontend` pod and one `wordpress-mysql` pod. Their IP addresses are in the 10.244.0.x range. Each pod shows 1 restart (12 minutes ago), which is consistent with the cluster restart in Figure 61. Nothing runs on `minikube-m02` yet, because the worker node was just added.

```
ubuntu@ip-172-31-38-26:~$ kubectl get pods -o wide
NAME                                READY   STATUS    RESTARTS      AGE   IP            NODE       NOMINATED NODE   READINESS GATES
nginx-598cc96cd9-jhf89              1/1     Running   1 (12m ago)   30h   10.244.0.12   minikube   <none>           <none>
nginx-dep-5fd6b8bd65-m98rl          1/1     Running   1 (12m ago)   29h   10.244.0.2    minikube   <none>           <none>
nginx-dep-5fd6b8bd65-qx9tf          1/1     Running   1 (12m ago)   28h   10.244.0.4    minikube   <none>           <none>
nginx-dep-5fd6b8bd65-w6jw8          1/1     Running   1 (12m ago)   29h   10.244.0.11   minikube   <none>           <none>
nginx-dep-5fd6b8bd65-z5q95          1/1     Running   1 (12m ago)   29h   10.244.0.8    minikube   <none>           <none>
todo-deployment-79ddc7dc7-9nmcq     1/1     Running   1 (12m ago)   29h   10.244.0.10   minikube   <none>           <none>
todo-deployment-79ddc7dc7-nh5r6     1/1     Running   1 (12m ago)   29h   10.244.0.7    minikube   <none>           <none>
todo-deployment-79ddc7dc7-rlcj4     1/1     Running   1 (12m ago)   29h   10.244.0.9    minikube   <none>           <none>
wordpress-frontend-6c76bd44-bwcx8   1/1     Running   1 (12m ago)   29h   10.244.0.6    minikube   <none>           <none>
wordpress-mysql-5bc89547d7-tm76v    1/1     Running   1 (12m ago)   29h   10.244.0.5    minikube   <none>           <none>
ubuntu@ip-172-31-38-26:~$
```

Figure 67: *All ten application pods running on the original node minikube*

16. Scheduling the To-Do Replicas on the Worker Node

By default the Kubernetes scheduler may place a pod on any Ready node. To place the three to-do replicas deliberately on the worker node, a node selector is added to the pod template of `todo-deployment`. The Nginx and WordPress deployments are not changed.

16.1 Re-checking the Nodes Before Scheduling

Purpose

The worker node must be Ready before pods are directed to it, and its exact node name is needed for the node selector.

Command used

```
kubectl get nodes -o wide
```

Command explanation

- `kubectl get nodes -o wide` — lists the nodes with full details; this capture uses a wide terminal, so all columns appear on one line.

Observed result

Both nodes are Ready and run version v1.37.0. `minikube` is the control-plane node (age 30h, IP 192.168.49.2) and `minikube-m02` is the added node (age 8m16s, IP 192.168.49.3, role <none>). Both show Debian GNU/Linux 12 (bookworm), kernel 7.0.0-1006-aws (amd64) and containerd://2.3.4, and EXTERNAL-IP is <none> for both. The node name `minikube-m02` is the name used in the next command.

```
ubuntu@ip-172-31-38-26:~$ kubectl get nodes -o wide
NAME           STATUS   ROLES           AGE     VERSION   INTERNAL-IP    EXTERNAL-IP   OS-IMAGE                         KERNEL-VERSION           CONTAINER-RUNTIME
minikube       Ready    control-plane   30h     v1.37.0   192.168.49.2   <none>        Debian GNU/Linux 12 (bookworm)   7.0.0-1006-aws (amd64)   containerd://2.3.4
minikube-m02   Ready    <none>          8m16s   v1.37.0   192.168.49.3   <none>        Debian GNU/Linux 12 (bookworm)   7.0.0-1006-aws (amd64)   containerd://2.3.4
```

Figure 68: *Node details confirming that minikube-m02 is Ready before scheduling*

16.2 Adding a Node Selector to the To-Do Deployment

Purpose

The patch tells Kubernetes that the to-do pods may only run on the node `minikube-m02`.

Commands used

```
kubectl patch deployment todo-deployment \
 -p
'{"spec":{"template":{"spec":{"nodeSelector":{"kubernetes.io/hostname":"minikube-
m02"}}}}}'
```

Command explanation

- `kubectl patch deployment todo-deployment` — changes the existing deployment in place without editing its YAML file.
- `-p` — supplies the patch as JSON.
- `"nodeSelector":{"kubernetes.io/hostname":"minikube-m02"}` — added to the pod template; a pod is scheduled only on a node whose built-in `kubernetes.io/hostname` label is `minikube-m02`.
- The trailing backslash only continues the command on the next line.

Observed result

Kubernetes answers `deployment.apps/todo-deployment patched`, so the change was accepted. Because the pod template changed, the Deployment replaces its pods with new ones that follow the selector, as the next figure shows. The screenshots do not show the file deployment.yml from Part 1 being edited; the change was made to the live Deployment.

```
ubuntu@ip-172-31-38-26:~$ kubectl patch deployment todo-deployment \
  -p '{"spec":{"template":{"spec":{"nodeSelector":{"kubernetes.io/hostname":"minikube-m02"}}}}}'
deployment.apps/todo-deployment patched
```

Figure 69: *Patching todo-deployment with a node selector for minikube-m02*

16.3 Watching the Rollout

Purpose

The patch starts a rolling update, and I wait until it has finished before checking the pods.

Command used

```
kubectl rollout status deployment/todo-deployment --timeout=180s
```

Command explanation

- `kubectl rollout status deployment/todo-deployment` – follows the update of the deployment and prints its progress.
- `--timeout=180s` – stops waiting after 180 seconds if the rollout has not finished.

Observed result

The output reports that 1 of 3 new replicas, then 2 of 3 new replicas, have been updated, followed by messages that 1 old replica is pending termination. It ends with `deployment "todo-deployment" successfully rolled out`. The old pods were therefore replaced gradually by new ones, and the rollout completed within the timeout.

```
ubuntu@ip-172-31-38-26:~$ kubectl rollout status deployment/todo-deployment --timeout=180s
Waiting for deployment "todo-deployment" rollout to finish: 1 out of 3 new replicas have been updated...
Waiting for deployment "todo-deployment" rollout to finish: 1 out of 3 new replicas have been updated...
Waiting for deployment "todo-deployment" rollout to finish: 1 out of 3 new replicas have been updated...
Waiting for deployment "todo-deployment" rollout to finish: 2 out of 3 new replicas have been updated...
Waiting for deployment "todo-deployment" rollout to finish: 2 out of 3 new replicas have been updated...
Waiting for deployment "todo-deployment" rollout to finish: 2 out of 3 new replicas have been updated...
Waiting for deployment "todo-deployment" rollout to finish: 2 out of 3 new replicas have been updated...
Waiting for deployment "todo-deployment" rollout to finish: 1 old replicas are pending termination...
Waiting for deployment "todo-deployment" rollout to finish: 1 old replicas are pending termination...
Waiting for deployment "todo-deployment" rollout to finish: 1 old replicas are pending termination...
deployment "todo-deployment" successfully rolled out
```

Figure 70: *Rollout of todo-deployment completed successfully*

16.4 Verifying the New Pod Placement

Purpose

This is the key check of the scheduling change: it shows on which node each pod now runs.

Command used

```
kubectl get pods -o wide
```

Command explanation

- `kubectl get pods -o wide` — lists all pods together with their IP address and node.

Observed result

The three `todo-deployment` pods (prefix 79cb5b74fb: `6m8d2`, `8ngg7` and `w6b28`) are 1/1 Running with 0 restarts and ages of 15 to 31 seconds, and the NODE column shows `minikube-m02` for all three. Their IP addresses are 10.244.1.4, 10.244.1.3 and 10.244.1.2, a different range from the 10.244.0.x addresses of the pods on `minikube`. The old to-do pods are gone. The `nginx` pod, the four `nginx-dep` pods, `wordpress-frontend` and `wordpress-mysql` are unchanged and still run on `minikube`. This shows the required placement: Todo on the worker node, Nginx and WordPress on the original node.

```
ubuntu@ip-172-31-38-26:~$ kubectl get pods -o wide
NAME                              READY   STATUS    RESTARTS      AGE   IP            NODE           NOMINATED NODE   READINESS GATES
nginx-598cc96cd9-jhf89            1/1     Running   1 (18m ago)   30h   10.244.0.12   minikube       <none>           <none>
nginx-dep-5fd6b8bd65-m98rl        1/1     Running   1 (18m ago)   30h   10.244.0.2    minikube       <none>           <none>
nginx-dep-5fd6b8bd65-qx9tf        1/1     Running   1 (18m ago)   29h   10.244.0.4    minikube       <none>           <none>
nginx-dep-5fd6b8bd65-w6jw8        1/1     Running   1 (18m ago)   29h   10.244.0.11   minikube       <none>           <none>
nginx-dep-5fd6b8bd65-z5q95        1/1     Running   1 (18m ago)   30h   10.244.0.8    minikube       <none>           <none>
todo-deployment-79cb5b74fb-6m8d2  1/1     Running   0             15s   10.244.1.4    minikube-m02   <none>           <none>
todo-deployment-79cb5b74fb-8ngg7  1/1     Running   0             17s   10.244.1.3    minikube-m02   <none>           <none>
todo-deployment-79cb5b74fb-w6b28  1/1     Running   0             31s   10.244.1.2    minikube-m02   <none>           <none>
wordpress-frontend-6c76bd44-bwcx8 1/1     Running   1 (18m ago)   29h   10.244.0.6    minikube       <none>           <none>
wordpress-mysql-5bc89547d7-tm76v  1/1     Running   1 (18m ago)   29h   10.244.0.5    minikube       <none>           <none>
ubuntu@ip-172-31-38-26:~$
```

Figure 71: *The three to-do pods running on minikube-m02 while the other pods remain on minikube*

17. Verifying the Two-Node Cluster and the Services

I re-check the nodes and then review the pods and Services some minutes after the change.

17.1 Node Status After the Rollout

Purpose

This check confirms that the worker node is still healthy after the to-do pods were moved to it.

Command used

```
kubectl get nodes -o wide
```

Command explanation

- `kubectl get nodes -o wide` — lists the nodes with their status and details.

Observed result

`minikube` (control-plane, age 30h) and `minikube-m02` (age 15m, role <none>) are both Ready with version v1.37.0 and internal IPs 192.168.49.2 and 192.168.49.3. The capture is cut off at the right edge, so only the first columns are visible; the node status, version and IP addresses are fully shown.

```
ubuntu@ip-172-31-38-26:~$ kubectl get nodes -o wide
NAME           STATUS   ROLES           AGE   VERSION   INTERNAL-IP    EXTERNAL-IP   OS-IMAGE                         KERNEL-VERSION
minikube       Ready    control-plane   30h   v1.37.0   192.168.49.2   <none>        Debian GNU/Linux 12 (bookworm)   7.0.0-1006-aws (amd
minikube-m02   Ready    <none>          15m   v1.37.0   192.168.49.3   <none>        Debian GNU/Linux 12 (bookworm)   7.0.0-1006-aws (amd
ubuntu@ip-172-31-38-26:~$
```

Figure 72: *Both nodes still Ready after the to-do pods were moved*

17.2 Reviewing the Pods and the Services

Purpose

The pod list confirms that the workloads are stable, and the Service list shows how the applications are exposed.

Commands used

```
kubectl get pods -o wide
kubectl get services
```

Command explanation

- `kubectl get pods -o wide` → lists the pods with their IP address and node.
- `kubectl get services` → lists the Services with their type, cluster IP, external IP, ports and age.

Observed result

All ten pods are 1/1 Running. The three to-do pods are on `minikube-m02` with ages of about 7 to 8 minutes and 0 restarts; the other seven pods are on `minikube`. The Service list shows six Services. `kubernetes` (cluster IP 10.96.0.1, port 443/TCP) and `wordpress-mysql` (cluster IP 10.111.191.104, port 3306/TCP) are of type ClusterIP, so they are reachable only inside the cluster. `nginx` (80:30131/TCP) and `todo-service` (cluster IP 10.105.145.167, 80:30007/TCP) are of type NodePort, which publishes the Service on a port of the nodes. `nginx-svc` (80:30405/TCP) and `wordpress-frontend` (80:31766/TCP) are of type LoadBalancer, but their EXTERNAL-IP is <pending>: no external load balancer has been assigned to them in this Minikube setup, so they have no external load-balancer IP. A Service selects its pods by label, not by node, so `todo-service` keeps routing to the to-do pods wherever they run.

```
ubuntu@ip-172-31-38-26:~$ kubectl get pods -o wide
NAME                              READY   STATUS    RESTARTS      AGE     IP            NODE           NOMINATED NODE   READINESS GATES
nginx-598cc96cd9-jhf89            1/1     Running   1 (25m ago)   30h     10.244.0.12   minikube       <none>           <none>
nginx-dep-5fd6b8bd65-m98rl        1/1     Running   1 (25m ago)   30h     10.244.0.2    minikube       <none>           <none>
nginx-dep-5fd6b8bd65-qx9tf        1/1     Running   1 (25m ago)   29h     10.244.0.4    minikube       <none>           <none>
nginx-dep-5fd6b8bd65-w6jw8        1/1     Running   1 (25m ago)   30h     10.244.0.11   minikube       <none>           <none>
nginx-dep-5fd6b8bd65-z5q95        1/1     Running   1 (25m ago)   30h     10.244.0.8    minikube       <none>           <none>
todo-deployment-79cb5b74fb-6m8d2  1/1     Running   0             7m39s   10.244.1.4    minikube-m02   <none>           <none>
todo-deployment-79cb5b74fb-8ngg7  1/1     Running   0             7m41s   10.244.1.3    minikube-m02   <none>           <none>
todo-deployment-79cb5b74fb-w6b28  1/1     Running   0             7m55s   10.244.1.2    minikube-m02   <none>           <none>
wordpress-frontend-6c76bd44-bwcx8 1/1     Running   1 (25m ago)   29h     10.244.0.6    minikube       <none>           <none>
wordpress-mysql-5bc89547d7-tm76v  1/1     Running   1 (25m ago)   29h     10.244.0.5    minikube       <none>           <none>
ubuntu@ip-172-31-38-26:~$ kubectl get services
NAME              TYPE           CLUSTER-IP       EXTERNAL-IP   PORT(S)        AGE
kubernetes        ClusterIP      10.96.0.1        <none>        443/TCP        30h
nginx             NodePort       10.111.198.135   <none>        80:30131/TCP   30h
nginx-svc         LoadBalancer   10.109.177.192   <pending>     80:30405/TCP   30h
todo-service      NodePort       10.105.145.167   <none>        80:30007/TCP   29h
wordpress-frontend LoadBalancer  10.111.210.19    <pending>     80:31766/TCP   29h
wordpress-mysql   ClusterIP      10.111.191.104   <none>        3306/TCP       29h
ubuntu@ip-172-31-38-26:~$
```

Figure 73: *Pods and Services after the to-do pods were moved to the worker node*

18. Accessing the To-Do Application

The to-do application is made reachable from a browser again through port forwarding, as in Part 1.

18.1 Starting the Port Forward

Purpose

The to-do Service is not exposed through an external IP, so `kubectl port-forward` is used to publish it on a port of the EC2 instance.

Command used

```
kubectl port-forward service/todo-service 8083:80 --address=0.0.0.0
```

Command explanation

- `kubectl port-forward service/todo-service` — forwards traffic to the Service `todo-service`.
- `8083:80` — local port 8083 on the EC2 instance is forwarded to port 80 of the Service.
- `--address=0.0.0.0` — listens on all network interfaces of the EC2 instance instead of only on localhost, so a browser on another computer can connect (port 8083 is allowed in the security group, Figure 5).

Observed result

The command prints `Forwarding from 0.0.0.0:8083 -> 8000`, so port 8083 is forwarded to the container port 8000 behind the Service. There is no new prompt below it because the command keeps running; the terminal session has to stay open while the application is used.

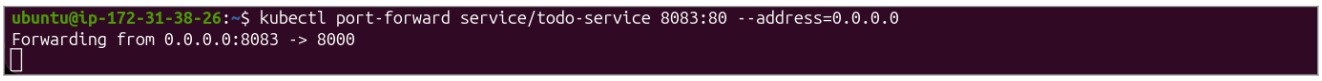


Figure 74: *Port forwarding of todo-service to port 8083*

18.2 Opening the To-Do Application in the Browser

Purpose

The browser test shows whether the application answers through the Service now that its pods run on the worker node.

Command used

```
http://56.228.34.11:8083/todos/ (address typed in the browser)
```

Command explanation

- `56.228.34.11` — the public IPv4 address of the EC2 instance at the time of this test (see Figure 80).
- `8083` — the forwarded port; `/todos/` — the path of the to-do page.

Observed result

The browser shows the page “Todo List - Modified once again” with an input field (“Do laundry”), an Add button and two tasks, “Send Resume Google now !” and “Hacktoberfest Updates”, each with a delete icon and both unchecked. Chrome marks the page “Not secure” because it is served over plain HTTP. All three to-do pods run on `minikube-m02`, so the application is served from the worker node. The address 56.228.34.11 differs from the public IP in Part 1; Figure 80 explains this.

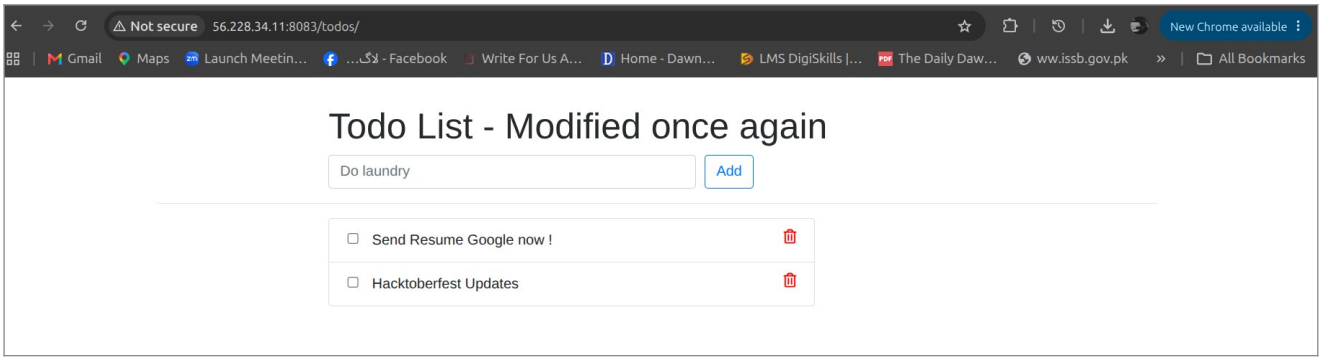


Figure 75: *To-do application opened in the browser at 56.228.34.11:8083/todos/*

19. Final Node Verification and Self-Healing Test

Finally, I check the nodes again and then test whether Kubernetes replaces a deleted to-do pod.

19.1 Checking the Node Roles and Readiness

Purpose

This check confirms that both nodes are Ready and shows the role of the worker node.

Command used

```
kubectl get nodes
```

Command explanation

- `kubectl get nodes` – lists the nodes with their status, role, age and version.

Observed result

`minikube` is Ready with the role control-plane (age 30h) and `minikube-m02` is Ready with the role worker (age 41m); both run v1.37.0. In the earlier captures (Figures 65, 68 and 72) the role of `minikube-m02` was <none>. kubectl takes the ROLES value from a node label of the form `node-role.kubernetes.io/<role>`, so a worker role label must have been added to `minikube-m02` between those captures. The command that did this is not part of the supplied screenshots, so it is not reproduced here. The scheduling of the to-do pods does not depend on this label; it depends on the `kubernetes.io/hostname` node selector.

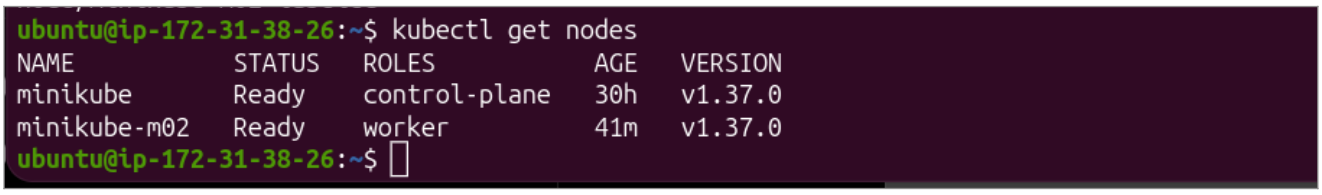


Figure 76: *Both nodes Ready; minikube-m02 now shown with the role worker*

19.2 Deleting a To-Do Pod

Purpose

Deleting one pod simulates the failure of a single replica, to see whether Kubernetes restores the desired number of three.

Command used

```
kubectl delete pod todo-deployment-79cb5b74fb-6m8d2
```

Command explanation

- `kubectl delete pod todo-deployment-79cb5b74fb-6m8d2` → deletes the pod with this name in the default namespace.

Observed result

Kubernetes confirms `pod "todo-deployment-79cb5b74fb-6m8d2" deleted from default namespace`. This is one of the three to-do pods that were created on `minikube-m02` in Figure 71.



Figure 77: *Deleting the to-do pod todo-deployment-79cb5b74fb-6m8d2*

19.3 Observing the Replacement Pod

Purpose

The pod list shows whether the Deployment has created a replacement for the deleted pod and where it runs.

Command used

```
kubectl get pods -o wide -w
```

Command explanation

- `kubectl get pods -o wide` → lists the pods with their IP address and node.
- `-w` → watch mode; the command keeps running and prints any later change to the pods.

Observed result

The deleted pod `6m8d2` is no longer listed. A new pod, `todo-deployment-79cb5b74fb-d5ktl`, is 1/1 Running with an age of 2m36s and IP 10.244.1.5, and its NODE is `minikube-m02`. The other two to-do pods, `8ngg7` (36m) and `w6b28` (37m), are unchanged, so there are three Running to-do replicas again, all on the worker node. The seven Nginx, WordPress and MySQL pods are unchanged on `minikube`. The capture shows the result after the replacement; it does not show the moment of creation. The replacement pod follows the node selector, so the Deployment restored the replica count and the placement.

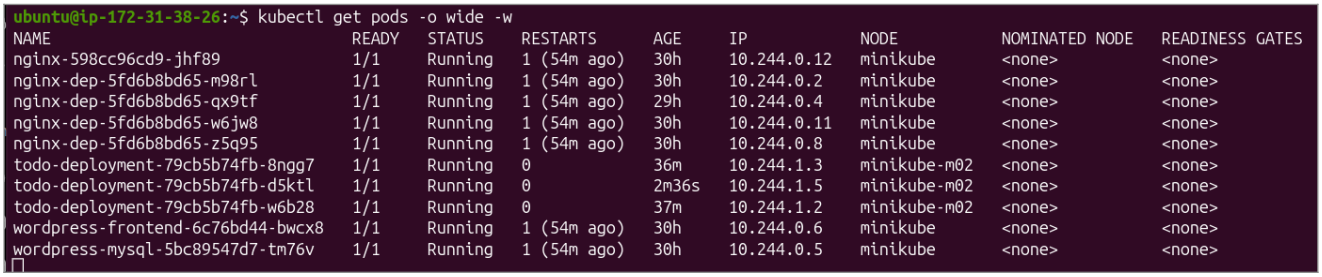


Figure 78: *Replacement pod d5ktl running on minikube-m02 after the deletion*

This test shows that the Deployment replaces an individual deleted pod. It is not a test of a node or host failure. Both Kubernetes nodes run as containers on the same EC2 instance, so if that instance stopped, both nodes and all pods would stop together. Recovery from a worker-node or EC2 failure was not tested in this lab.

19.4 Checking the To-Do Application in the Browser

Purpose

A last browser check shows that the application is still reachable through the forwarded port.

Command used

```
http://56.228.34.11:8083/todos/ (address typed in the browser)
```

Command explanation

- The address is the same forwarded address as in Figure 75.

Observed result

The to-do page “Todo List - Modified once again” is displayed again at 56.228.34.11:8083/todos/. This time the task “Send Resume Google now !” is ticked and shown struck through, while “Hacktoberfest Updates” is unticked, so the page responds and shows the task state. In the order of the source screenshots, this capture follows the pod-replacement test.

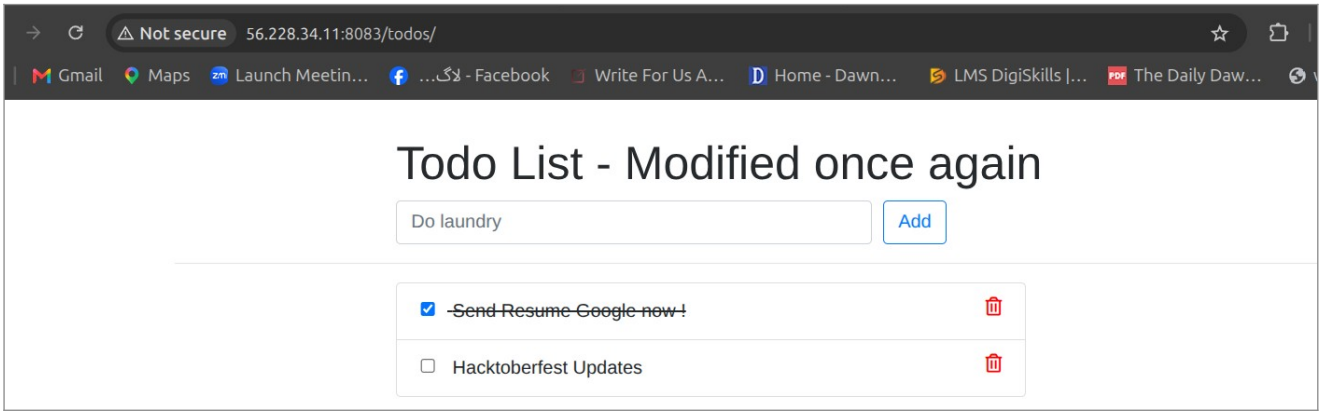


Figure 79: *To-do application still answering in the browser, with one task marked as completed*

19.5 Confirming the EC2 Instance and Its Public IP Address

Purpose

The EC2 console is checked to explain the public IP address used in the browser tests of Part 2 and to confirm that both nodes belong to one instance.

Command used

```
(AWS EC2 console: instance summary page, no command)
```

Command explanation

- The summary page lists the instance ID, state, public and private IPv4 addresses and public DNS name.

Observed result

The summary is for instance i-0e2030e9ad8ded0d6 (sami_kubernetes_lab), which is in the Running state. Its public IPv4 address is 56.228.34.11 (public DNS ec2-56-228-34-11.eu-north-1.compute.amazonaws.com) and its private IPv4 address is 172.31.38.26. The instance ID and private address are the same as in Part 1, so this is the same EC2 instance, but the public IPv4 address differs from the 16.171.154.62 shown in Part 1 (Figure 4). It matches the address used in Figures 75 and 79. The screenshots do not show why the address changed; an automatically assigned public IPv4 address on EC2 normally changes when an instance is stopped and started.

Because the private address is unchanged and only one instance is shown, both Minikube nodes run on this single EC2 instance.

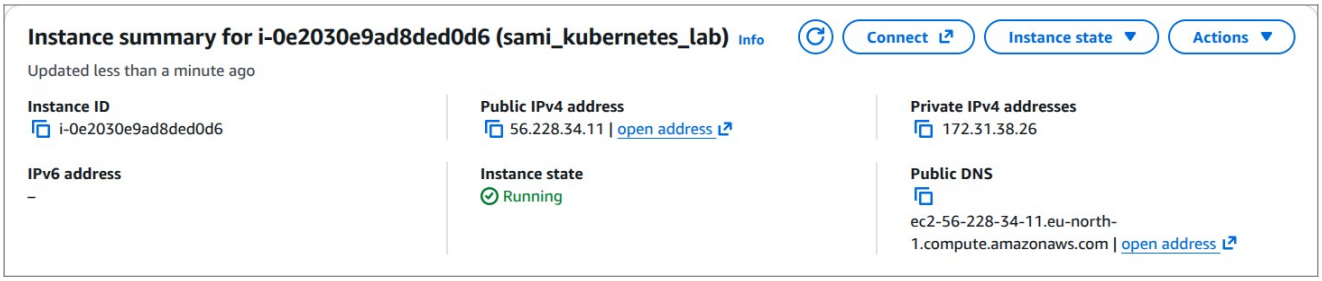


Figure 80: *EC2 instance summary showing the current public IPv4 address 56.228.34.11*

20. Conclusion

In Part 1 of this lab, a Kubernetes cluster was created with Minikube on an Ubuntu AWS EC2 instance, after installing Docker, Minikube and kubectl. Nginx was deployed with kubectl commands and with YAML manifests, a WordPress and MySQL application and a to-do list application were deployed, and all applications were made reachable from a browser using `kubectl port-forward`. The tests in Sections 8.6 and 12.3 showed that Kubernetes automatically replaces a deleted pod to keep the desired number of replicas. The table below summarises the applications deployed in Part 1.

Application	Kubernetes objects	Service	Browser port
Nginx (kubectl commands)	Deployment nginx (1 replica)	nginx, NodePort 80:30131	8082
Nginx (YAML manifests)	Deployment nginx-dep (4 replicas)	nginx-svc, LoadBalancer 80:30405	8080
WordPress + MySQL	Deployments wordpress-frontend and wordpress-mysql (1 replica each)	wordpress-frontend, LoadBalancer 80:31766; wordpress-mysql, ClusterIP 3306	8081
To-do list	Deployment todo-deployment (3 replicas)	todo-service, NodePort 80:30007	8083

In Part 2, the single-node cluster was extended to a two-node logical Minikube cluster with `minikube node add --worker`. The cluster then consisted of the control-plane node `minikube` and the worker node `minikube-m02`, both Docker containers on the same EC2 instance, so the cluster is not made of two separate EC2 instances. Both nodes were verified as Ready with `minikube node list` and `kubectl get nodes -o wide`. A node selector on `kubernetes.io/hostname: minikube-m02` was patched into `todo-deployment`, and the rolling update moved all three to-do replicas to the worker node, while the Nginx, WordPress and MySQL pods stayed on `minikube`. The table below summarises the workload placement shown by the pod listings.

Workload	Node	Pods running	Figures
Nginx (kubectl commands)	minikube	1 (nginx)	67, 71, 73
Nginx (YAML manifests)	minikube	4 (nginx-dep)	67, 71, 73
WordPress frontend	minikube	1 (wordpress-frontend)	67, 71, 73
WordPress MySQL	minikube	1 (wordpress-mysql)	67, 71, 73
To-do list (Django)	minikube-m02	3 (todo-deployment), 3 again after pod deletion	71, 73, 78

`kubectl get services` showed ClusterIP, NodePort and LoadBalancer Services; the two LoadBalancer Services have the external IP <pending>, because no external load balancer is provisioned in this Minikube setup. The to-do application was reached through `kubectl port-forward` on port 8083 and opened in the browser, and the node lists, pod lists and browser pages showed that both nodes were Ready and the application pods were Running. When one to-do pod was deleted, the Deployment created a replacement that was again scheduled on `minikube-m02`, which confirms self-healing at the level of an individual pod.

Overall, the lab demonstrates how Kubernetes orchestrates containerised applications. Deployments describe the desired number of replicas and replace pods that are deleted; Services give pods a stable address and expose them as ClusterIP, NodePort or LoadBalancer Services; and the scheduler places pods on nodes, which can be controlled with a node selector. The lab also shows a limit of the setup: replacing a single pod is not the same as surviving the failure of a node or host, and because both Minikube nodes run on one EC2 instance, the loss of that instance would stop the whole cluster. Recovery from a node or host failure was not tested.